



Generation AI

Nurturing Future Innovators

«ԱԲ սերունդ» ծրագիր Համակարգչային գիտություն

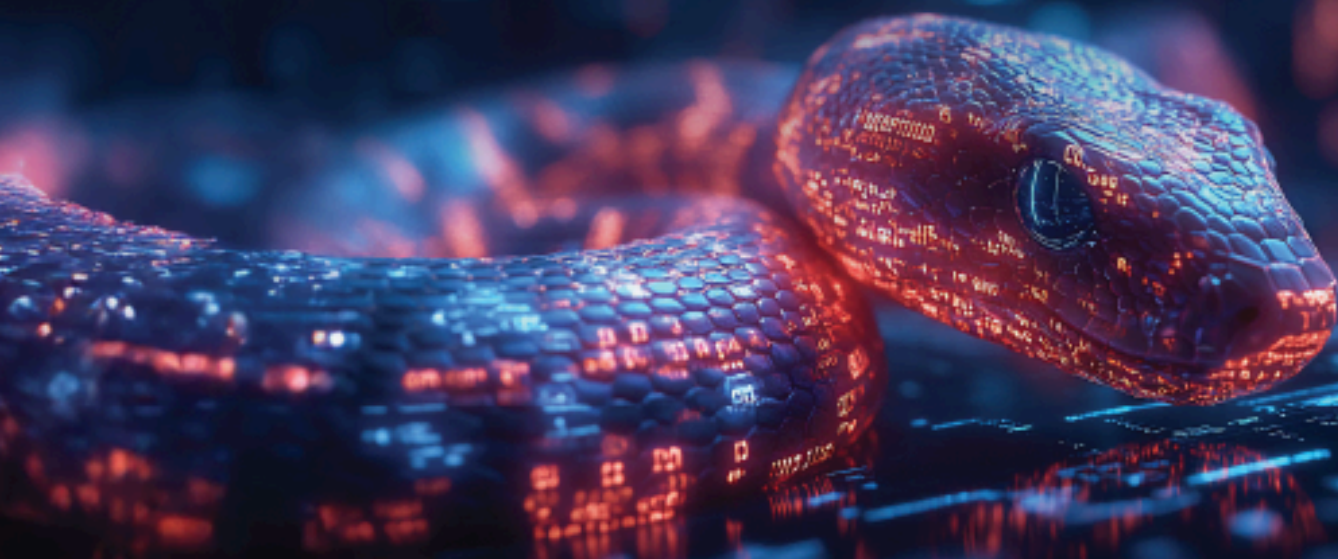
Python

Ծրագրավորման լեզվով

Դասագիրք

Տարբերակ 1.0

11-րդ դասարան



Այսօր ծրագրավորումն ընկալվում է ոչ միայն որպես գործնական հմտություն, այլև որպես համաշխարհային կրթական գրագիտության առանցքային բաղադրիչ: Python-ը՝ իր պարզ կառուցվածքով և ընթերցելիությամբ, համարվում է ամենամատչելի և բազմակողմանի ծրագրավորման լեզուներից մեկը՝ հնարավորություն ընձեռելով միջին և ավագ դասարանների սովորողներին վստահությամբ կատարել առաջին քայլերը ծրագրավորման աշխարհում:

ՀՀ հանրակրթության առարկայական չափորոշիչներով՝ Python-ի հիմունքները ներառված են «Թվային գրագիտություն և համակարգչային գիտություն» առարկայի շրջանակում՝ սկսած 7-րդ դասարանից, իսկ «ԱԲ սերունդ» ծրագրում այն նախատեսված է որպես համակարգչային գիտության հիմնական լեզու 10-րդ և 11-րդ դասարաններում: Այնուամենայնիվ, հայերենում առկա ուսումնական նյութերը սահմանափակ են. բացատրությունները հաճախ չափազանց բարդ և խրթին են, ինչի պատճառով դժվարանում են հասկանալ թե՛ ուսուցիչները, թե՛ աշակերտները:

Սույն դասագիրքը մշակվել է՝ հաշվի առնելով Python-ի դասավանդման երկար տարիների փորձը, ուսուցիչների և աշակերտների կողմից բարձրաձայնված կարիքները և միջազգային համակարգչային գիտության կրթական միտումները: Դասագիրքը նպատակ ունի պարզ ու հասկանալի բացատրություններով զարգացնել սովորողների կողմնորոշման և խնդիրներ լուծելու հմտությունները՝ տարրականից մինչև բարդ մակարդակ: Առաջադրանքները խրախուսում են քննադատական մտածողություն, գործնական մոտեցում և ինքնուսուցում՝ ձևավորելով ամուր հիմք հետագա խորացված ուսման և կիրառությունների համար:

Python ուսումնական դասագրքի մշակող-փորձագետ՝

Միհրան Ավագյան, ԳՏՃՄ մանկավարժ-փորձագետ, FAST հիմնադրամ, «ԱԲ սերունդ» ծրագրի վերջնարդյունքի վերլուծաբան

Python ուսումնական դասագրքի վերանայող փորձագետ՝

Ավագ Սայան, Տեխնոկրթության փորձագետ, FAST հիմնադրամ, «ԱԲ սերունդ» ծրագրի բովանդակային ղեկավար

Երախտիքի խոսքեր.

Մենք մեր երախտագիտությունն ենք հայտնում «ԱԲ սերունդ» ծրագրի շուրջ ձևավորված մասնագիտական համայնքին՝ մշակվող նյութերը պարբերաբար վերանայելու և բարելավմանն ուղղված դիտարկումներ տրամադրելու համար: Դասագրքի այս տարբերակը գտնվում է փորձարկման փուլում:

«Սույն նյութի, ներառյալ դրա ամբողջ բովանդակության, ձևավորման, տեքստերի, պատկերների և ցանկացած այլ տարրի հեղինակային իրավունքը պատկանում է Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամին:

Սույն նյութը չի կարող ամբողջությամբ կամ մասնակիորեն վերարտադրվել, տարածվել, փոխանցվել, վերախմբագրվել, թարգմանվել կամ որևէ այլ ձևով օգտագործվել առանց Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամի նախնական գրավոր համաձայնության՝ անկախ օգտագործման նպատակից: Սույն նյութի մեջ արտահայտված տեսակետները, վերլուծություններն ու մեկնաբանությունները հանդիսանում են հեղինակների մտավոր սեփականությունը:

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

ԹԵՄԱ 1: ԴԱՍԵՐ (CLASS).....	4
1. ԺԱՌԱՆԳՈՒՄ (INHERITANCE).....	8
2. ՊՈԼԻՄՈՐՖԻԶՄ (POLYMORPHISM).....	8
3. ԻՆԿԱՊՍՈՒԼԱՑԻԱ (ENCAPSULATION).....	8
4. ԱՔՍՏՐԱԿՑԻԱ (ABSTRACTION).....	9
ԹԵՄԱ 2. ՄՈԴՈՒԼՆԵՐ (MODULES).....	26
NUMPY ԳՐԱԴԱՐԱՆ.....	27
MATPLOTLIB PYPLOT ԳՐԱԴԱՐԱՆ.....	32
PANDAS ԳՐԱԴԱՐԱՆ.....	54
ԹԵՄԱ 3: ԱՅԼ ԳՐԱԴԱՐԱՆՆԵՐ: ՄԻՋԱՎԱՅՐԵՐ:.....	65
PLOTLY ԳՐԱԴԱՐԱՆ.....	69
ՀԻՄՆԱԿԱՆ ԱՌԱՋԱՅԱՑԱՏՎՈՒԹՅՈՒՆՆԵՐԸ.....	70
PLOTLY EXPRESS ՀՐԱՄԱՆՆԵՐԻ ԱՂՅՈՒՄԱԿ.....	70
ԹԵՄԱ 4: ՍԽԱԼՆԵՐԻ ՀԱՆԳՈՒՑԱԼՈՒԾՈՒՄ.....	86
ՍԽԱԼՆԵՐԻ ՏԵՍԱԿՆԵՐ.....	86
ՍԵՓԱԿԱՆ ԲԱՑԱՌՈՒԹՅՈՒՆՆԵՐԻ ՍՏԵՂԾՈՒՄ.....	89
ՎՐԻՊԱԶԵՐԾՄԱՆ ԳՈՐԾԻՔՆԵՐ, ԵՒ ՏՐԱՄԱԲԱՆԱԿԱՆ ՍԽԱԼՆԵՐԻ ՎԵՐԼՈՒԾՈՒԹՅՈՒՆ.....	91
ՍԽԱԼՆԵՐԻ ՀԱՂՈՐԴԱԳՐՈՒԹՅՈՒՆՆԵՐԻ ՍՏԵՂԾՈՒՄ USER-ՆԵՐԻ ՀԱՄԱՐ.....	95
ԹԵՄԱ 5: ՏԱՐԲԵՐԱԿԻ ՎԵՐԱՀՍԿՄԱՆ ՀԱՄԱԿԱՐԳ.....	98
GIT-Ի ՄԱՍԻՆ.....	98
GIT-Ի ՀԻՄՆԱԿԱՆ ՀՆԱՐԱՎՈՐՈՒԹՅՈՒՆՆԵՐԸ.....	98
GIT-Ի ՀԵՏ ԱԾԽԱՏԵԼՈՒ ՔԱՅԼԵՐԸ.....	98
ԳՐԱԿԱՆՈՒԹՅՈՒՆ ԵՎ ԱՐՏԱՔԻՆ ՀՂՈՒՄՆԵՐ.....	102

Թեմա 1: ԴԱՍԵՐ (CLASS)

ԹԵՄԱ 1: ԴԱՍԵՐ (CLASS)

Համակարգչային գիտության ոլորտում գոյություն ունեն ծրագրավորման տարբեր հայեցակարգեր, որոնք նախատեսված են՝ տարատեսակ խնդիրներ լուծելու համար: Մինչ class-ներին անցնելը եկե՛ք թվարկենք և սահմանենք հիմնական հայեցակարգերը:

- **Ֆունկցիոնալ ծրագրավորում (Functional programming)**

FP-ն կենտրոնանում է մաքուր մաթեմատիկական ֆունկցիաների (pure functions) և փոփոխականների սահմանափակման վրա: Այս տարբերակը հարմար է տարատեսակ մաթեմատիկական հաշվարկների և տվյալների վերլուծության համար: Ֆունկցիոնալ ծրագրավորմանն աջակցող լեզուներից են՝ Haskell-ը, JavaScript-ը, Python-ը, Scala-ն, Erlang-ը, Lisp-ը, ML-ը, Clojure-ը, OCaml-ը, Common Lisp-ը, Racket-ը և այլն:

- **Ընթացակարգային ծրագրավորում (Procedural programming, PP)**

PP-ն կենտրոնացած է ֆունկցիաների և հրահանգների հաջորդականության վրա: Երբեմն այն անվանում են 3GL, որը նշանակում է երրորդ սերոնի ծրագրավորման լեզու: Այն պահանջում է ծրագրավորողից կոդավորել ոչ միայն «ինչ անել»-ը, այլև «ինչպես անել»-ը: Հետևյալ ծրագրավորման լեզուները՝ FORTRAN, C, Basic, Pascal, ALGOL, COBOL, JAVA, հիմնականում օգտագործել են այդ հայեցակարգը: Նմանատիպ ծրագրավորումն այսօր գրեթե չի օգտագործվում, սակայն մի շարք երկրներում պետական համակարգի ծրագրային ապահովումներն աշխատում են այս լեզուներով և մեթոդով գրված ծրագրերով:

- **Ռեակտիվ ծրագրավորում (Reactive programming, RP)**

RP-ն կենտրոնանում է իրադարձություններով կառավարվող ծրագրավորման վրա: Հարմար է UI, ապարատողության և իրական ժամանակի տվյալների մշակման համար: Աջակցող լեզուներից են՝ Haskell-ը, Scala-ն, Erlang-ը, Elixir-ը, Java-ն, C# - ReactiveX-ը և այլն:

- **Առարկայա-կողմնորոշված ծրագրավորում (Object-oriented programming, OOP)**

OOP-ն կենտրոնանում է առարկաների վրա, որոնք պարունակում են տվյալներ (ատրիբուտներ) և գործողություններ (մեթոդներ): Հարմար է իրական աշխարհի մոդելավորման համար: Սա ամենաշատ օգտագործվող հայեցակարգն է և օգտագործվում է գրեթե բոլոր առաջադեմ ծրագրավորման լեզուներում (Java, C++, Python, C#):

Այս հայեցակարգերն ունեն մի շարք առավելություններ և թերություններ, դրանք զարգացել են տարբեր ժամանակաշրջաններում՝ փոխարինելով մեկը մյուսին: Փաստացի կարող ենք ընդունել որպես տրամաբանական զարգացում: Աղյուսակում ներկայացված են հայեցակարգերի իմաստաբանական տարբերությունները:

Functional	Procedural	Reactive	Object-oriented
Function	Procedure	Observable/Stream	Method
Persistent Data Str.	Record	Signal/Event	Object
Module/Namespace	Module	Observable Pipeline	Class
Function composition	Procedure call	Subscription	Message (Method call)
Immutable data	Global state	Time-varying state	Instance state
Recursion	Loops	Operators (map, filter)	Inherit./Polymorphism
Values	Variables	Behavior / Subject	Properties
Pure functions	Side effects	Asynchronous events	Encapsulation

Եկե՛ք օրինակի վրա տեսնենք վերոնշյալ հայեցակարգերով գրված Python լեզվի ծրագրավորման կոդերի տարբերությունները, որոնք տպելիս նույն արդյունքն են տալիս:
Ունենք խնձոր, որի բնօրինակը կանաչ է, և կախված կարիքներից՝ հարկավոր է օգտագործել այն տարբեր գույներով:



Functional Programming (FP)

```
Python
def transform_apple(apple):
    """ Վերադարձնում է նոր
    փոխակերպված խնձորն առանց
    բնօրինակը փոփոխելու: """
    new_apple = {
        "size": apple["size"] * 2,
        "color": "red" if
apple["color"] == "green" else
"yellow"
    }
    return new_apple

# բնօրինակ խնձորը
original_apple = {"size": 5, "color":
"green"}

# ձևափոխում
transformed_apple =
transform_apple(original_apple)

print("Original Apple:",
original_apple)
print("Transformed Apple:",
transformed_apple)
```

Procedural Programming (PP)

```
Python
# խնձորի սկզբնական վիճակը
apple = {"size": 5, "color":
"green"}

# քայլ-առ-քայլ ձևափոխում
print("Step 1: Picking the
apple...")
apple["size"] += 2
print("Step 2: Changing the
color...")
apple["color"] = "red"
print("Step 3: Transformation
complete!")

print("Transformed Apple:", apple)
```

Reactive Programming (RP)

```
Python
import time

def transform_apple():
    print("Waiting for event (Press
Enter to transform the apple)...")
    input() # իրադարձություններն
    ըստ օգտատիրոջ
    apple["size"] *= 2
    apple["color"] = "red"
    print("The apple has
transformed:", apple)

# խնձորի սկզբնական վիճակը
apple = {"size": 5, "color":
"green"}

print("Original Apple:", apple)
transform_apple()
```

Object oriented Programming (OOP)

```
Python
class Apple:
    def __init__(self, size,
color):
        self.size = size
        self.color = color

    def transform(self):
        """խնձորի հատկությունները
        փոփոխելու մեթոդ. """
        self.size *= 2
        self.color = "red"
        print(f"Transformed Apple
-> Size: {self.size}, Color: |
{self.color}")

# ստեղծել խնձորի օբյեկտ
apple = Apple(size=5,
color="green")

print(f"Original Apple -> Size:
{apple.size}, Color:
{apple.color}")
apple.transform()
```

Մաքուր ֆունկցիաներ

Քայլեր

Իրադարձություններ

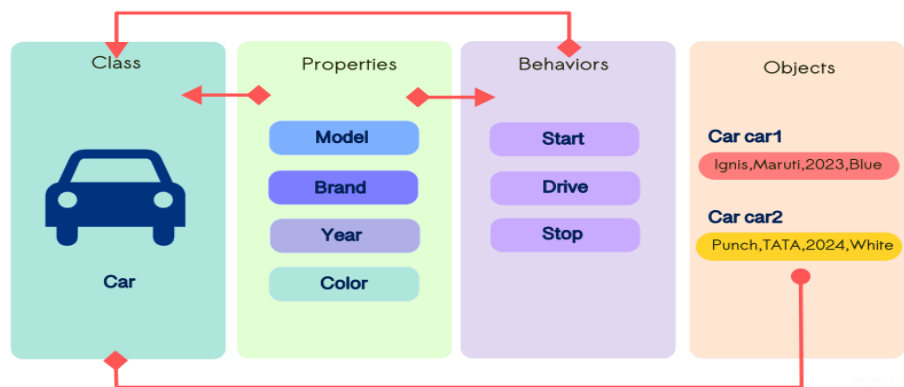
Ատրիբուտներ ու
մեթոդներ

Original Apple -> Size: 5, Color: green
Transformed Apple -> Size: 10, Color: red

Ծրագրավորման սկզբնական փուլերում բոլոր գործողություններն իրականացվում էին ֆունկցիաների միջոցով: Ֆունկցիաները թույլ էին տալիս կոդը մասերի բաժանել, վերագործել և դառնում էին ավելի կառավարելի: Սակայն, երբ ծրագրերը մեծացան, առաջացան մի շարք խնդիրներ, որոնք ֆունկցիաներով հնարավոր չէր լուծել: Թվարկենք հիմնական խնդիրները.

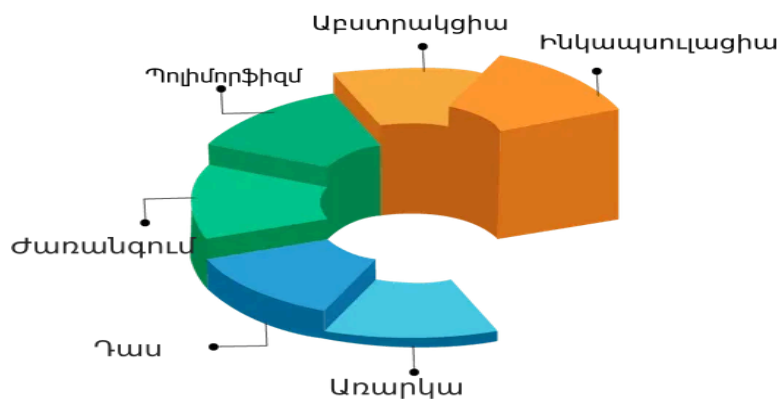
1. Տվյալների և գործառնությունների միջև կապի բացակայություն:
2. Ֆունկցիաների կոդմիջ տվյալները չէին պահպանվում:
3. Կոդի վերագործման սահմանափակում:
4. Կառավարման դժվարություն:

Այսպիսով՝ կարիք առաջացավ ներդնելու ավելի բարձր կառուցվածքային տարր՝ **class (դաս)**: Այն առարկայի «կադապար» է, որը նկարագրում է հատկությունները (properties) և մեթոդները (methods):



Նկար 1. Class-ի (դաս) օրինակ մեքենայի համար: Աղբյուր՝ [bigkno1](#):

Python լեզվում և առհասարակ առարկայա-կողմնորոշված ծրագրավորման (OOP) մեջ կարևոր դեր ունեն ժառանգում, պոլիմորֆիզմ և ինկապսուլացիա, աբստրակցիա հասկացությունները, քանի որ օգնում են կառուցել կազմակերպված, վերագործելի, համապարփակ և անվտանգ կոդեր: OOP-ն հիմնականում կենտրոնանում է առարկաների և class-երի վրա, սակայն ազդում է ամբողջ ծրագրի կառուցվածքի վրա: Ուստի, երբ ասում ենք OOP, ավելի շատ նկատի ունենք առարկան և class-ը:



1. Ժառանգում (inheritance)

Ժառանգումը թույլ է տալիս մեկ դասին (դուստր կամ ենթադաս) ժառանգել հատկություններ (attributes) և մեթոդներ (methods) մյուս դասից (ծնող դաս): Այսպիսով՝ նոր դասը կարող է օգտագործել նախապես գրված կոդը՝ առանց այն կրկնելու:

2. Պոլիմորֆիզմ (Polymorphism)

Բառն ունի հետևյալ բացատրությունը. պոլի նշանակում է շատ, իսկ մորֆ՝ ձև, ըստ այդմ՝ բազմաթիվ տվյալների ձևաչափում կամ կարգավորում՝ բազմաձևություն: Պոլիմորֆիզմի կարիքը ծագեց 1960-ականներին՝ առարկայա-կողմնորոշված ծրագրավորման (OOP) զարգացման փուլում: Սկսեցին միևնույն մեթոդն օգտագործել տարբեր առարկաների համար՝ հեշտացնելով կոդի վերաօգտագործումն ու ընդլայնումը: Հետևաբար՝ պոլիմորֆիզմը թույլ է տալիս տարբեր դասերին (class) ունենալ նույն անունով մեթոդներ, բայց կատարել տարբեր իրագործումներ: Այսպիսով՝ կարող ենք գրել ունիվերսալ կոդ, որը կաշխատի տարբեր օբյեկտների համար:



Նկար 3. Պոլիմորֆիզմի օրինակ իրական կյանքից: Աղբյուր՝ vividexamples

Նկար 1-ում պատկերված է թիթեռի զարգացման տարբեր փուլերի ձևափոխումները՝ թրթուրային ձևից մինչև ամբողջական հասուն միջատ: Այս զարգացումը նման է class-երի պոլիմորֆիզմին, քանի որ բոլոր փուլերում այն թիթեռ է, սակայն արտաքին տեսքերը տարբերվում են:

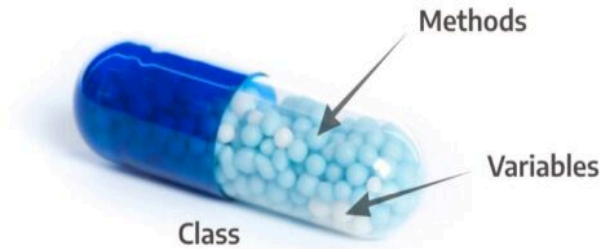
3. Ինկապսուլացիա (Encapsulation)

Բառն ունի հետևյալ բացատրությունը. Capsule՝ «կապսուլ» կամ պարկուճ և նշանակում է փակել, ծածկել կամ կաղապարել:



Նկար 4. Դեղահաբի պարկուճ, պատկերի աղբյուր՝ [wiki](#)

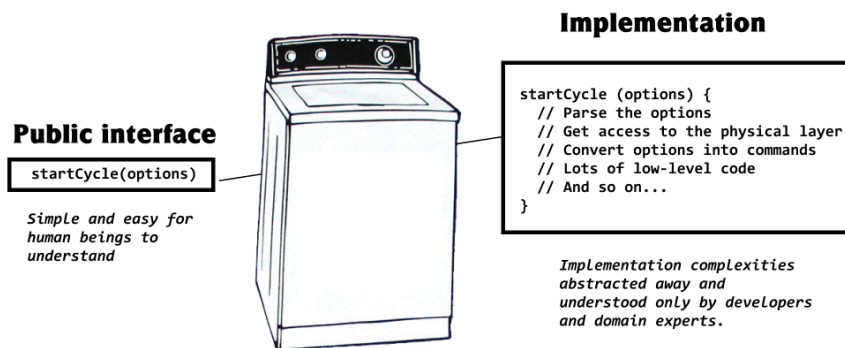
Ծրագրավորման լեզուներում (C, C++, Python) ինկապսուլացիան տվյալների պաշտպանման մեխանիզմ է, որը սահմանափակում է արտաքին միջավայրի տվյալներին անմիջապես հասանելիությունը: Սա ապահովվում է `private` (մասնավոր) փոփոխականների և մեթոդների միջոցով (`__` նախածանցով):



Նկար 5. Ինկապսուլացիան ծրագրավորման մեջ, պատկերի աղբյուր՝ [usemynotes](#):

4. Աբստրակցիա (Abstraction)

Աբստրակցիա բառը նշանակում է բացառում, անջատում, վերացում, վերացողականություն: Աբստրակցիայի գաղափարի անհրաժեշտությունն առաջացավ այն ժամանակ, երբ ծրագրավորումը դարձավ ավելի բարդ և բազմազան (1960-1970 թթ.): Մեծ ծրագրերում հարկավոր է լինում թաքցնել որոշ մասեր և օգտատերերին հասանելի դարձնել միայն անհրաժեշտության դեպքում:



Նկար 6. Աբստրակցիան իրական կյանքում, պատկերի աղբյուր՝ [khalilstemmler](#):

ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ստեղծի՛ր **Student** դասը (class):

Ատրիբուտներ	Մեթոդներ
- name` անուն,</li> <li>- <b>age</b>` տարիք, - grade` դասարան:	- introduce(), որը կտալի՝ "Ես [անուն] եմ, [տարիք] տարեկան եմ և սովորում եմ [դասարան]-րդ դասարանում:"

2. Ստեղծի՛ր **Car** դասը:

Ատրիբուտներ	Մեթոդներ
- brand` արտադրող,</li> <li>- <b>model</b>` մոդել, - year` արտադրության տարեթիվ:	- car_info()` վերադարձնում է մեքենայի արտադրողի և մոդելի անվանումը,</li> <li>- <b>age(current_year)</b>` վերադարձնում է մեքենայի տարիքը՝ կախված ընթացիկ տարվանից:

3. Ստեղծի՛ր **FootballPlayer** դասը:

Ատրիբուտներ	Մեթոդներ
- name` խաղացողի անուն,</li> <li>- <b>team</b>` խաղացողի ներկա թիմը, - position` խաղացողի դիրքը:	- player_info()` տալում է խաղացողի տվյալները,</li> <li>- <b>change_team(new_team)</b>` փոխում է խաղացողի թիմը:

4. Ստեղծի՛ր **Book** դասը:

Ատրիբուտներ	Մեթոդներ
- title` գրքի վերնագիր,</li> <li>- <b>author</b>` հեղինակ, - genre` ժանր,</li> <li>- <b>pages</b>` էջերի քանակը, - publisher` հրատարակչություն,</li> <li>- <b>publication_year</b>` տպագրման տարեթիվ:	- get_description()` վերադարձնում է գրքի կարճ նկարագրությունը` վերնագրով և հեղինակով, - sheet_count()` վերադարձնում է գրքի թերթերի քանակը,</li> <li>- <b>age(current_year)</b>` հաշվում է գրքի տարիքը` ելնելով տպագրման տարեթվից:

5. Ստեղծի՛ր **Animal** դասը:

Ատրիբուտներ	Մեթոդներ
- age` տարիք,</li> <li>- <b>weight</b>` քաշ, - speed` արագություն:	- info()` վերադարձնում է կենդանու բոլոր տվյալները,</li> <li>- <b>update_speed(n)</b>` կենդանու արագությունը դարձնում է n, - change_weight(a)` փոխում է կենդանու քաշը a-ով:

- Ստեղծի՛ր **Cat** դասը, որը **կժառանգի** animal դասից:

Նոր ատրիբուտներ	Նոր մեթոդներ
- color` գույն,</li> <li>- <b>species</b>` տեսակ (օրինակ` siamese, bengal):	- get_age_category()` գտնում և տպում է. <ul style="list-style-type: none"> <li>➢ կատուն երիտասարդ է` տարիքը փոքր է 7-ից, ➢ միջին տարիքի է` 7-ից 11 տարեկան,</li> <li>➢ ծեր է` 11-ից բարձր:

6. Ստեղծիր **Car** դասը:

Ատրիբուտներ	Մեթոդներ
- brend` արտադրող,</li> <li>- <b>model</b>` մոդել, - year` արտադրման տարեթիվ,</li> <li>- <b>max_speed</b>` առավելագույն արագությունը կմ/ժ-ով, - color` գույն:	- info()` տպում է մեքենայի բոլոր տվյալները,</li> <li>- <b>change_color (color)</b>` փոխում է մեքենայի գույնը: - age (current_year)` հաշվում է մեքենայի տարիքը:

- Ստեղծիր **Electro_car** ենթադասը, որը **ժառանգում** է **Car** դասից:

Նոր ատրիբուտներ	Նոր մեթոդ
- battery_size` մարտկոցի չափը կՎտժ,</li> <li>- <b>charging_time</b>` լիցքավորման ժամանակը (րոպե):	- price()` հաշվում է ամբողջական լիցքավորման արժեքը (1 կՎտ.ժ-ի արժեքը 120 դրամ):

- Ստեղծիր **Hydrogen_car** ենթադասը, որը **ժառանգում** է **Car** դասից:

Նոր ատրիբուտներ	Նոր մեթոդ
- hydrogen_tank_capacity` մեքենայի բակի տարողությունը (կգ),</li> <li>- <b>current_hydrogen_level</b>` մեքենայի ընթացիկ վառելիքի ծավալը, որը պետք է փոքր կամ հավասար լինի բակի տարողությանը, - range_per_kg` քանի կմ կարող է անցնել 1 կգ ջրածնով:	- refuel(amount)` լրացնում է ջրածնի բաքը amount կգ-ով, բայց ոչ ավել քան <b>hydrogen_tank_capacity</b>-ն,</li> <li>- <b>calculate_range()</b>` հաշվարկում է, թե քանի կմ կարող է անցնել ներկա ջրածնի մակարդակով, - check_fuel_status()` վերադարձնում է, թե քանի % ջրածին է մնացել բաքում:

7. Ստեղծիր **Animal** դասը:

Ատրիբուտներ	Մեթոդներ
- age` տարիք,</li> <li>- <b>weight</b>` քաշ, - speed` արագություն:	- info()` վերադարձնում է կենդանու բոլոր տվյալները:</li> <li>- <b>update_speed(n)</b>` կենդանու արագությունը դարձնում է n: - change_weight(a)` փոխում է կենդանու քաշը <b>a</b>-ով:</li> <li>- <b>make_sound()</b>` վերադարձնում է <i>"unknown"</i> տեքստ:

- Ստեղծիր **Cat**, **Dog** և **Cow** ենթադասեր, որնք **ժառանգում** են **Animal** դասից, իսկ **make_sound()** մեթոդը վերադարձնում է հետևյալ ձայները (պոլիմորֆիզմ)`

Dog - "Woof-woof"

Cat - "Meow"

Cow - "Moo"

- Ստեղծիր **Fish** ենթադասը, որը ժառանգում է **Animal** դասից, սակայն չի ունենում **make_sound()** մեթոդը:

8. Ստեղծիր **CreditCardPayment** դաս:

Ատրիբուտներ	Մեթոդներ
- card_number` քարտի համար,</li> <li>- <b>card_holder</b>` քարտապանի անունը, - balance` գումարի մնացորդը:	- pay(amount)` կատարում է վճարում, եթե հաշվի մնացորդը թույլ է տալիս և տպում է (պոլիմորֆիզմ). <i>Paid \$50 using Credit Card (1234-5678-9876-5432):</i>

- Ստեղծի՛ր **PayPalPayment** դասը:

Ատրիբուտներ	Մեթոդներ
- email` օգտատիրոջ PayPal էլ. փոստը,</li> <li>- <b>balance</b>` գումարի մնացորդ:	- pay(amount)` կատարում է վճարում, եթե հաշվի մնացորդը թույլ է տալիս և տպում է. <i>Paid \$30 using PayPal (john@example.com)</i>:

9. Ստեղծի՛ր **Square** դասը:

Ատրիբուտ	Մեթոդներ
- length` կողմի երկարություն:	- area()` հաշվարկում է քառակուսու մակերեսը,</li> <li>- <b>perimeter()</b>` հաշվարկում է քառակուսու պարագիծը:

- Ստեղծի՛ր **Triangle** դասը:

Ատրիբուտներ	Մեթոդներ
- side1` եռանկյան առաջին կողմ,</li> <li>- <b>side2</b>` եռանկյան երկրորդ կողմ, - side3` եռանկյան երրորդ կողմ:	- is_void()` ստուգում է նշված կողմերով եռանկյուն կա, թե ոչ,</li> <li>- <b>area()</b>` հաշվում է եռանկյան մակերեսը, - perimeter()` հաշվարկում է եռանկյան պարագիծը:

- Ստեղծի՛ր **Circle** դասը:

Ատրիբուտ	Մեթոդներ
- radius` շառավիղ:	- circumference()` շրջանագծի երկարություն,</li> <li>- <b>area()</b>` շրջանի մակերես:

10. Ստեղծի՛ր **Person** դասը:

Ատրիբուտներ	Մեթոդներ
- name` անուն (private),</li> <li>- <b>age</b>` տարիք (private), - id` անձնագրի համար (private):	- get_info()` վերադարձնում է անունը, տարիքը,</li> <li>- <b>get_older()</b>` տարիքը մեծացնում է 1-ով, - get_id()` վերադարձնում է անձնագրի համարը:

11. Ստեղծի՛ր **BankAccount** դասը (ինկապսուլացիա):

Ատրիբուտներ	Մեթոդներ
- balance` հաշվի մնացորդ (private),</li> <li>- <b>password</b>` գաղտնաբառ (private):	- deposit(amount, password)` ավելացնում է գումար հաշվին. գումարը պետք է դրական լինի, իսկ գաղտնաբառը` ճիշտ, - withdraw(amount, password)` հանում է գումար հաշվից, amount-ը չպետք է հաշվի մնացորդից մեծ լինի, իսկ գաղտնաբառը պետք է ճիշտ լինի,</li> <li>- <b>change_password(old_password, new_password)</b>` փոխում է գաղտնաբառը, եթե հին գաղտնաբառը ճիշտ է գրված, - get_balance(password)` վերադարձնում է հաշվի մնացորդը, եթե գաղտնաբառը ճիշտ է:

12. Ստեղծի՛ր WeatherStation դասը:

Ատրիբուտներ	Մեթոդներ
- <code>humidity</code>` խոնավություն (private),</li> <li>- <code>pressure</code>` մթնոլորտային ճնշում (private), - <code>temperature</code>` ջերմաստիճան (private):	- <code>get_humidity()</code>` վերադարձնում է խոնավությունը,</li> <li>- <code>get_pressure()</code>` վերադարձնում է մթնոլորտային ճնշումը, - <code>get_temperature()</code>` վերադարձնում է ջերմաստիճանը,</li> <li>- <code>update_measure(tem,hum,pre)`</code> թարմացնում է ջերմաստիճանի, խոնավության և ճնշման տվյալները, - <code>average_temperature()</code>` վերադարձնում է միջին ջերմաստիճանը,</li> <li>- <code>average_humidity()</code>` վերադարձնում է միջին խոնավությունը, - <code>average_pressure()</code>` վերադարձնում է միջին մթնոլորտային ճնշումը,</li> <li>- <code>max_temperature(), min_temperature()</code>` վերադարձնում է առավելագույն / նվազագույն ջերմաստիճանները, - <code>max_humidity(), min_humidity()</code>` վերադարձնում է առավելագույն / նվազագույն խոնավությունը,</li> <li>- <code>max_pressure(), min_pressure()</code>` վերադարձնում է առավելագույն, նվազագույն մթնոլորտային ճնշումը, - <code>draw_scatter()</code>` կազմում է ցրված (scatter plot) դիագրամ ջերմաստիճանի կախումը խոնավությունից,</li> <li>- <code>last_5_measurements()</code>` վերադարձնում է վերջին 5 չափումների տվյալները: Եթե տվյալները քիչ են 5-ից, ապա տպում է բոլոր տվյալները:

13. Ստեղծիր **Library** դասը:

Ատրիբուտ	Մեթոդներ
- book` բառարաններից կազմված ցուցակ է` {title: str, author: str, available: bool} (private):	- add_book(title, author)` ավելացնում է նոր գիրք գրադարանի բազայում,</li> <li>- <b>remove_book(title)</b>` հեռացնում է գիրքը գրադարանից, - borrow_book(title)` նշում է, որ գիրքը տեղում չէ և հասանելիությունը դառնում է False,</li> <li>- <b>return_book(title)</b>` նշում է, որ գիրքը վերադարձվել է գրադարան, - list_available_books()` վերադարձնում է բոլոր հասանելի գրքերը,</li> <li>- <b>search_by_author(author)</b>` վերադարձնում է տվյալ հեղինակին պատկանող բոլոր գրքերը, - save_to_file(file_name)` ստեղծում է file_name անունով txt ֆայլ և վրան պահում է գրքերի ցանկը:

14. Ստեղծիր **GPS_Tracker** դաս:

Ատրիբուտներ	Մեթոդներ
- latitude` լայնություն (private),</li> <li>- <b>longitude</b>` երկայնություն (private):	- move(new_latitude, new_longitude)` փոխում է օբյեկտի դիրքը,</li> <li>- <b>current_location()</b>` վերադարձնում է ընթացիկ դիրքի կոորդինատները, - distance()` վերադարձնում է օբյեկտի վերջին տեղափոխության երկարությունը,</li> <li>- <b>all_location()</b>` վերադարձնում է օբյեկտի եղած բոլոր դիրքերի կոորդինատները, - distance_from_origin()` վերադարձնում է սկզբնակետի և ներկա դիրքի հեռավորությունը,</li> <li>- <b>higher()</b>` վերադարձնում է ներկա դիրքի բարձրությունը ծովի մակարդակից: -

Kotlin

```
import requests

def get_elevation(latitude, longitude):
    # Օգտագործում ենք OpenElevation API
    url =
f"https://api.open-elevation.com/api/v1/lookup?locations={latitude},{l
ongitude}"

    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        elevation = data['results'][0]['elevation']
        return elevation
    else:
        return "Error: Could not retrieve elevation data."

# GPS կոորդինատների օրինակ
latitude = 28.596111 # Վշխարհագրական լայնություն
longitude = 83.820274 # Վշխարհագրական երկայնություն

elevation = get_elevation(latitude, longitude)
print(f"Բարձրությունը ծովի մակարդակից` {elevation} մետր:") #նւնի
որոշակի սխալանքի չափ
```

15. Ստեղծի՛ր HEV դաս ([Hybrid Electric Vehicle](#)):

Ատրիբուտներ	Մեթոդներ
- brand` արտադրող,</li> <li>- <b>model</b>` մոդել, - fuel_efficiency` վառելիքի արդյունավետությունը (Էլեկտրաէներգիայի հետ միաժամանակ 100 կմ-ի համար օգտագործվող վառելիքի քանակը),</li> <li>- <b>battery_level</b>` մարտկոցի մակարդակ (0-100%), - fuel_capacity` վառելիքի քանակը (առավելագույնը 50լ):	- info ()` վերադարձնում է մեքենայի բոլոր տվյալները,</li> <li>- <b>refuel (amount)</b>` ավելացնում է վառելիքի ծավալը amount-ով, եթե լիցքավորումից հետո, այն չի գերազանցում 50 լիտրը:

- Ստեղծի՛ր **PHEV** ենթադասը ([Plug-in Hybrid Electric Vehicle](#)):

Ատրիբուտներ*	Նոր մեթոդներ
- Ժառանգում է HEV դասից` բացառությամբ fuel_efficiency ատրիբուտի:	- fuel_cost` ցույց է տալիս առանց Էլեկտրաէներգիայի բենզինի վառելիքի ծախսը 100 կմ-ի համար,</li> <li>- <b>charge(procent)</b>` լիցքավորում է մեքենան համապատասխան տոկոսով:

- Ստեղծի՛ր **BEV** ենթադասը (Battery Electric Vehicle): Այս դասը կժառանգի PHEV դասից` բացառությամբ **fuel_capacity**, **fuel_cost** ատրիբուտներից և **refuel()** մեթոդից:

16. Ստեղծի՛ր **SmartPhone** դասը:

Ատրիբուտներ	Մեթոդներ
- <code>brand</code>` արտադրող,</li> <li>- <code>model</code>` մոդել, - <code>price</code>` գինը,</li> <li>- <code>color</code>` գույնը:	- <code>make_call(number)</code>` տպում է հետևյալ հաղորդագրությունը. “<i>Ձանգ է կատարվում [number] համարին</i>”:</li> <li>- <code>send_message(number, message)</code>` տպում է հետևյալ հաղորդագրությունը. “<i>Հաղորդագրությունն [message] ուղարկվել է [number] համարին</i>”:

- Ստեղծի՛ր `AndroidPhone` ենթադասը:

Լրացուցիչ ատրիբուտ	Լրացուցիչ մեթոդ
- <code>android_version</code>` android-ի տարբերակ:	- <code>install_app(app_name)</code>` տպում է հետևյալ հաղորդագրությունը. «<i>[app_name] հավելվածը ներբեռնված է</i>»:

- Ստեղծի՛ր `Iphone` ենթադասը:

Լրացուցիչ ատրիբուտ	Լրացուցիչ մեթոդ
- <code>ios_version</code>` iOS-ի տարբերակը:	- <code>use_siri(command)</code>` տպում է հետևյալ հաղորդագրությունը. «<i>siri-ն կատարում է [command] հրամանը</i>»:

- Ստեղծի՛ր `GamingPhone` ենթադասը:

Լրացուցիչ ատրիբուտներ	Լրացուցիչ մեթոդներ
- <code>cooling_system</code>` True կամ False, արդյո՞ք ունի հովացման համակարգ,</li> <li>- <code>game_mode</code>` սկզբում լռելայն ընդունում է False արժեքը:	- <code>change_mode ()</code>` եթե խաղային ռեժիմը միացած է, անջատում է, իսկ եթե անջատած է, միացնում է: Այսինքն՝ <code>game_mode</code>-ը փոխում է հակառակ արժեքի:</li> <li>- <code>boost ()</code>` կախված <code>game_mode</code>-ի արժեքից՝ տալում է “խաղային ռեժիմը միացված է” կամ “խաղային ռեժիմը անջատված է” հաղորդագրություններից մեկը:

17. Ստեղծի՛ր `GraphPlotter` դասը:

Ատրիբուտներ	Մեթոդներ
- <code>title</code>` վերնագիր,</li> <li>- <code>x_data</code>` սկզբում դատարկ ցուցակ է, - <code>y_data</code>` սկզբում դատարկ ցուցակ է,</li> <li>- <code>color</code>` գույն:	- <code>add_point (x,y)</code>` ավելացնում է նոր կետ,</li> <li>- <code>avg_data()</code>` վերադարձնում է x և y տվյալների միջինը, - <code>scatter_plot ()</code>` գծում է x, y տվյալների ցրված գրաֆիկը,</li> <li>- <code>change_color(new_color)</code>` փոխում է գույնը, - <code>save_scatter_plot(path)</code>` պահպանում է scatter plot-ը՝ որպես նշված ուղիով նկար:

18. Ստեղծիր `TemperatureAnalyzer` դասը:

Ատրիբուտներ	Մեթոդներ
- <code>days</code>` օրեր, սկզբում <code>numpy</code>-ի դատարկ զանգված է,</li> <li>- <code>temperatures</code>` ջերմաստիճաններ սկզբում <code>numpy</code>-ի դատարկ զանգված է, - <code>unit</code>` ջերմաստիճանի միավորը (“°C” կամ “°F”):	- <code>load_data_from_file(filename)</code>` ներմուծում է օրերի և ջերմաստիճանների տվյալները <code>csv</code> ֆայլից,</li> <li>- <code>temperature_scatter_plot ()</code>` տպում է ցրված գրաֆիկը, - <code>hottest_day()</code>` վերադարձնում է գրանցված ամենաբարձր ջերմաստիճանը,</li> <li>- <code>coldest_day()</code>` վերադարձնում է գրանցված ամենացածր ջերմաստիճանը, - <code>temperature_histogram()</code>` կառուցում է ջերմաստիճանների հիստոգրամը,</li> <li>- <code>temperature_mean()</code>` հաշվում է միջինը, - <code>temperature_median()</code>` հաշվում է մեդիանը,</li> <li>- <code>temperature_var()</code>` հաշվում է վարիացիան, - <code>temperature_std()</code>` հաշվում է ստանդարտ շեղումը,</li> <li>- <code>convert_to_fahrenheit()</code>` ջերմաստիճանները <code>temperature</code> զանգվածում փոխակերպում է ֆարենհայտի, - <code>convert_to_celsius()</code>` ջերմաստիճանները <code>temperature</code> զանգվածում փոխակերպում է ցելսիուսի:

19. Ստեղծիր **Runner** դասը:

Ատրիբուտներ	Մեթոդներ
- name` պահում է մարզիկի անունը:	- add_run(self, day, time)` ավելացնում է մարզման օրը (<b>day</b>) և վազքի ժամանակը:</li> <li>- <b>run_line_plot(self, color)</b>` line plot-ով պատկերում է մարզիկի ցույց տված արդյունքները, - run_time_mean(self)` հաշվում է վազքի միջին ժամանակը:</li> <li>- <b>best_run(self)</b>` գտնում է լավագույն արդյունքը (ամենաքիչ ժամանակով վազքը):

20. Ստեղծիր աբստրակտ **Shape** class-ը, որը կունենա աբստրակտ **area()** և **perimeter()** մեթոդները: Այնուհետև, ստեղծիր հետևյալ դասերը.
- **Rectangle**, որի **area()** մեթոդը կվերադարձնի մակերեսը, իսկ **perimeter()** մեթոդը` պարագիծը:
 - **Trinagle**, որի **area()** մեթոդը կվերադարձնի մակերեսը, իսկ **perimeter()** մեթոդը` շրջանագծի երկարությունը:

Python

```
from abc import ABC, abstractmethod
import math

# Միջնահարկի class
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

# Ուղղանկյունի class
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# Եռանկյունի class
class Trinagle (Shape):
    def __init__(self, side1,side2,side3):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3
```

ԹԵՄԱ 2:

ՄՈԴՈՒԼՆԵՐ
(MODULES)

ԹԵՄԱ 2. ՄՈԴՈՒԼՆԵՐ (MODULES)

PIP ՓԱԹԵԹՆԵՐԻ ԿԱՌԱՎԱՐՄԱՆ ՀԱՄԱԿԱՐԳ

pip-ի միջոցով հնարավոր է Python-ի համար տեղադրել, ջնջել, թարմացնել և ցուցադրել փաթեթները: pip փաթեթը տեղադրվում է Python ծրագրի հետ, եթե այն հատուկ չանջատենք (python3, pip3):

Հրաման	Նկարագրություն	Օրինակ
<code>pip install package</code>	Տեղադրել փաթեթ:	<code>pip install numpy</code>
<code>pip install package==1.2.3</code>	Տեղադրել կոնկրետ տարբերակ:	<code>pip install django==4.2</code>
<code>pip install --upgrade package</code>	Թարմացնել փաթեթը:	<code>pip install --upgrade pandas</code>
<code>pip uninstall package</code>	Ջնջել փաթեթը:	<code>pip uninstall flask</code>
<code>pip list</code>	Ցուցադրել բոլոր տեղադրված փաթեթները:	<code>pip list</code>
<code>pip show package</code>	Տեղադրված փաթեթի մասին ինֆորմացիա	<code>pip show matplotlib</code>
<code>pip freeze > requirements.txt</code>	Պահել փաթեթների ցանկը ֆայլում:	<code>pip freeze > requirements.txt</code>
<code>pip install -r requirements.txt</code>	Տեղադրել requirements.txt-ից:	<code>pip install -r requirements.txt</code>
<code>python -m pip install --upgrade pip</code>	Թարմացնել pip-ը:	<code>python -m pip install --upgrade pip</code>

NumPy ԳՐԱԴԱՐԱՆ

NumPy-ն (Numerical Python) ստեղծել է Թրեվիս Օլիֆենթը 2005 թ.: Նա վերցրեց ավելի հին՝ Numeric (ստեղծվել էր 1995թ-ին) և Numarray, գրադարաններն ու համատեղեց դրանց հնարավորությունները մեկ հզոր գրադարանում: Օլիֆենթի նպատակը Python-ում արագ և արդյունավետ թվաբանական հաշվարկների իրականացումն էր մեծ զանգվածների և մատրիցների վրա: NumPy-ը լուծում էր հետևյալ խնդիրները՝



Նպաստի արդյունքը՝ numbarray.com
Թրեվիս Օլիֆենթ

- մեծ տվյալների պահպանում և մշակման օպտիմալացում,
- արագ հաշվարկներ՝ օգտագործելով C լեզվով գրված ներքին մեխանիզմներ,
- հարմար array-ներ և մատրիցային գործողությունների API-ներ,
- հիմք այլ գիտական գրադարանների համար (SciPy, Pandas, Matplotlib և այլն):

NumPy-ի առավելությունները

- Արագություն՝ շատ ավելի արագ է իրականացնում գործողություններ մեծ զանգվածների վրա, քան Python-ի սովորական list-երը:
- Կոմպակտություն՝ array-ները հիշողության մեջ պահվում են օպտիմալացված:
- Հարմար API՝ տրամադրում է հարյուրավոր մաթեմատիկական ֆունկցիաներ:
- Ինտեգրում՝ շատ գիտական գրադարաններ հիմնված են NumPy-ի վրա:
- Համայնք և փաստաթղթավորում՝ լայնորեն օգտագործվում և զարգանում է:
- Համատեղելիություն C/C++/Fortran-ի հետ՝ թույլ է տալիս արագացնել հաշվարկները:

NumPy-ի կիրառություններ

- Google՝ օգտագործում է NumPy-ը TensorFlow-ի և AI նախագծերի հիմքում:
- Microsoft՝ օգտագործվում է տվյալների վերլուծության, Azure Machine Learning-ի և գիտական հաշվարկների համար:
- Amazon՝ ապահովում է տվյալների վերլուծություն, AI, AWS ծառայություններ:
- Meta (Facebook)՝ մեծ տվյալների և AI մոդելների համար:
- Netflix՝ տվյալների վերլուծություն, առաջարկությունների համակարգեր:
- Spotify, YouTube՝ օգտատերերի վերլուծություն, առաջարկությունների համակարգեր:
- Stripe, PayPal՝ վճարումների վերլուծություն, մեքենայական ուսուցում:

Հիմնական կառուցվածքներ

- NumPy-ի հիմնական կառուցվածքը կոչվում է զանգված (*array*), որը միանման տիպի տարրերի հավաքածու է: Չանգվածները շատ ավելի արդյունավետ են հիշողության և պրոցեսորի առումով, քան Python-ի բազային ցուցակները:
- 1D զանգվածներ՝ վեկտորների նման միաչափ զանգվածներ, օրինակ՝
[1, 2, 3, 4],
- 3D զանգվածներ (մատրիցներ)՝ օրինակ՝ [[1, 2], [3, 4]],

- 3D զանգվածներ՝ անսահմանափակ չափերի կառուցվածքներ:

M մատրիցի տարրերին կարելի է հասնել դրանց ինդեքսներով: Ինդեքսավորումը սկսում է 0-ից: Օրինակ՝ ունենք հետևյալ M մատրիցը՝

5	11	15
6	8	10
3	7	9

Առաջին տողի երկրորդ տարրին՝ 11-ին, կարելի է հասնել հետևյալ կերպ՝ $M[0, 1]$, կամ երրորդ տողի երկրորդ տարրին՝ 7-ին, կարելի է հասնել հետևյալ կերպ՝ $M[2, 1]$:

Չանգվածի ստեղծում

- `np.array([1, 2, 3])`՝ ստեղծում է զանգված՝ նշված արժեքներով:
- `np.zeros((3, 3))`՝ ստեղծում է 3×3 զրոների մատրից:
- `np.ones((2, 2))`՝ ստեղծում է 2×2 մեկերի մատրից:
- `np.arange(0, 10, 2)`՝ ստեղծում է 0 – ից 10 միջակայքում թվերի հերթականությունը 2 քայլով:
- `np.linspace(0, 1, 5)`՝ ստեղծում է 0 – ից 1 միջակայքի 5 հավասարապես բաշխված թվեր:
- `np.eye(3)`՝ ստեղծում է 3×3 միավոր մատրից:

Մաթեմատիկական օպերացիաներ

- `np.add(a, b)`՝ գումարում է a-ն և b-ն:
- `np.subtract(a, b)`՝ հանում է b-ն a-ից:
- `np.multiply(a, b)`՝ տարր առ տարր բազմապատկում է a-ն և b-ն:
- `np.divide(a, b)`՝ տարր առ տարր բաժանում է a-ն b-ով:
- `np.sum(A)`՝ գումարում է A մատրիցը:

Հատուկ ֆունկցիաներ

- `np.exp(a)`՝ հաշվում է e^a :
- `np.sqrt(a)`՝ հաշվում է a-ի քառակուսի արմատը:
- `np.log(a)`՝ հաշվում է բնական լոգարիթմը:
- `np.sin(a)`, `np.cos(a)`՝ հաշվում է a-ի սինուսը և կոսինուսը:

Գծային հանրահաշվային գործողություններ

- `np.dot(a, b)`՝ գտնում է մատրիցների a և b կրկնապատկման արդյունքը:
- `np.linalg.det(a)`՝ հաշվում է քառակուսի մատրիցի որոշիչը:
- `np.linalg.inv(a)`՝ հաշվում է մատրիցի հակադարձը (եթե մատրիցան հակադարձելի է):
- `np.linalg.eig(a)`՝ հաշվում է քառակուսի մատրիցի *eigen* արժեքները և *eigen* վեկտորները:
- `np.linalg.solve(a, b)`՝ լուծում է գծային հավասարումների համակարգը՝
 Օրինակ՝
$$\begin{aligned} 2x + 3y &= 8 \\ 3x + 4y &= 11 \end{aligned}$$

Լուծենք այս համակարգը՝

```
Python
import numpy as np

# Գծային հավասարումների գործակիցների մատրիցա
A = np.array([[2, 3], [3, 4]])

# Ազատ անդամների սյունը
B = np.array([8, 11])

# Լուծում ենք  $Ax = B$  համակարգը
solution = np.linalg.solve(A, B)

print("x և y արժեքները", solution)
```

Վիճակագրության տարրեր

- `np.mean(a)`՝ հաշվում է միջինը:
 - `np.median(a)`՝ հաշվում է մեդիանը:
 - `np.var(a)`՝ հաշվում է վարիացիան:
 - `np.std(a)`՝ հաշվում է ստանդարտ շեղումը:
 - `np.sum(a)`՝ հաշվում է տարրերի գումարը:
 - `np.min(a)`, `np.max(a)`՝ գտնում է նվազագույնն ու առավելագույնը:
 - `np.corrcoef(x, y)[1, 0]`՝ հաշվում է x և y տվյալների [կորրելացիան](#):
 - `np.cov(x, y)[1, 0]`՝ հաշվում է x և y տվյալների [կովարացիան](#):
- Կորելացիան և կովարացիան հաշվելիս տալիս է 2×2 չափանի մատրից, որտեղ՝
- `[0, 0]` տարրը X -ի փոփոխականի վարիացիան/կորրելացիան է,
 - `[1, 1]` տարրը Y -ի փոփոխականի վարիացիան/կորրելացիան է,
 - `[0, 1]` և `[1, 0]` տարրերը ցույց են տալիս X և Y -ի միջև կովարիացիան/կորրելացիան:

Տրամաբանական օպերատորներ

- `np.where(condition, x, y)`՝ վերադարձնում է x -ը, եթե պայմանը բավարարվում է, հակառակ դեպքում՝ y :
- `np.any(a)`՝ ստուգում է՝ արդյոք զանգվածում կա գոնե մեկ `True` արժեք:
- `np.all(a)`՝ ստուգում է՝ արդյոք բոլոր արժեքները `True` են:

Ձևափոխումներ

- `a.reshape((rows, cols))`՝ փոփոխում է զանգվածի չափսերը՝ առանց տվյալները փոխելու:
- `a.flatten(a)`՝ վերածում է բազմաչափ զանգվածը միաչափի:

- `np.transpose(a)`՝ շրջում է մատրիցը:

Տարրերի տեսակավորում և որոնում

- `np.unique(data)`՝ վերադարձնում է միայն եզակի տարրերից կազմված ցուցակ:
- `np.sort(a)`՝ տեսակավորում է զանգվածը:
- `np.argsort(a)`՝ վերադարձնում է տեսակավորման ինդեքսները:
- `np.argmax(a)`, `np.argmin(a)`՝ գտնում է առավելագույն և նվազագույն տարրերի ինդեքսները:

Պատահական թվերի գեներացում

- `np.random.random((3, 3))`՝ $-1, 1$ միջակայքում ստեղծում է պատահական արժեքներով 3×3 չափանի մատրից:
- `np.random.randint(-10, 10, (3, 3))`՝ $-10 -$ ից 10 միջակայքում ստեղծում է պատահական ամբողջ թվերով 3×3 չափանի մատրից:
- `np.random.normal(mean, std, size)`՝ ստեղծում է նորմալ բաշխված պատահական արժեքների զանգված:

Այս բաժնի խնդիրները լուծելիս պարտադիր է օգտվել *NumPy* գրադարանից:

Մի քանի թվերի ներմուծումները կազմակերպի՛ր ցիկլերի միջոցով, օրինակ՝ *for* ցիկլով:

1. Ներմուծի՛ր 5 թիվ, հաշվի՛ր այդ թվերի միջինը:

Լուծում

```
Python
import numpy as np
L=[]
for i in range(5):
    n = int ( input ("Ներմուծել թիվը"))
    L.append(n)
# Սահմանում ենք տվյալների զանգվածը
data = np.array(L)

# Հաշվում ենք միջինը
mean_value = np.mean(data)
print("Միջինը", mean_value)
```

2. Ստեղծի՛ր թվային ցուցակ: Գրի՛ր ծրագիր, որը բառարանով վերադարձնում է այդ ցուցակի մեծագույն, փոքրագույն և միջին արժեքները, վարիացիան, ստանդարտ շեղումը և մեդիանը:
3. Ստեղծի՛ր թվային ցուցակ: Գրի՛ր ծրագիր, որը տպում է այդ ցուցակի եզակի տարրերից կազմված ցուցակը: Օգտվի՛ր `np.unique` ֆունկցիայից:
4. Ունենք 10 նորածին երեխայի քաշերը՝ `[2.9, 3.3, 3.1, 3.6, 2.8, 3.5, 4.0, 3.2, 3.8, 2.7]`: Հաշվի՛ր նորածինների զանգվածի միջինը և ստանդարտ շեղումը:

5. Նորածինների քաշերի՝ [2.9, 3.3, 3.1, 3.6, 2.8, 3.5, 4.0, 3.2, 3.8, 2.7], տվյալների համար հաշվի՛ր վարիացիայի գործակիցը՝ $cv = \frac{\text{ստանդարտ շեղում}}{\text{տվյալների միջին}} \times 100$:
6. Հողագործները մշակում են այգին: Նրանց աշխատաժամերի և մշակված հողակտորի մակերեսները ներկայացված են հետևյալ աղյուսակով՝

X ժամ	1	2	3	4	5
Y հա	3	6	9	12	15

Գտի՛ր X և Y փոփոխականների միջև կովարիացիան և կորելացիան:

7. Ստեղծի՛ր [2, 0, 1, 3] զանգվածը և դրա վրա կիրառի՛ր *np.sqrt* ֆունկցիան: Կստանանք Նոր զանգված, որի յուրաքանչյուր անդամ Նախորդ անդամի զանգվածի քառակուսի արմատն է:
8. Ստեղծի՛ր [2, 0, 1, 3] զանգվածը և դրա վրա կիրառի՛ր *np.exp* ֆունկցիան: Կստանանք Նոր զանգված, որի յուրաքանչյուր անդամ Էսպոնենցիալ ֆունկցիայի արժեքն է Նախորդ զանգվածի համապատասխան կետում:
9. Ստեղծի՛ր 4×4 չափանի միավոր մատրից: Տպի՛ր այդ մատրիցը:
10. Ստեղծի՛ր 4×4 չափանի մատրից՝ [0, 100] միջակայքի պատահական թվերով: Տպի՛ր այդ մատրիցը և դրա որոշիչը:
11. Ստեղծի՛ր 2×3 և 3×4 չափանի պատահական մատրիցներ՝ կազմված 0-ից 10 միջակայքի ամբողջ թվերից: Տպի՛ր այդ մատրիցները և դրանց արտադրյալը:
12. Ստեղծի՛ր 3×3 չափանի երկու պատահական մատրից, տպի՛ր այդ մատրիցները և դրանց գումարը:
13. NumPy գրադարանով լուծի՛ր հետևյալ գծային հավասարումների համակարգը՝

$$3x + 2y = 7$$

$$x - y = 1$$

14. NumPy գրադարանով լուծի՛ր հետևյալ գծային հավասարումների համակարգը՝

$$x + 2y + 3z = 9$$

$$2x + 3y + z = 82$$

$$2x + y + 2z = 7$$

15. Ստեղծի՛ր 3×3 չափանի մատրից՝ [0, 100] միջակայքի պատահական թվերով: Տպի՛ր այդ մատրիցը և դրա տրանսպոնացված մատրիցը:
16. Ստեղծի՛ր 3×3 չափանի մատրից՝ [0, 100] միջակայքի պատահական թվերով: Տպի՛ր այդ մատրիցի և դրա տրանսպոնացված մատրիցի որոշիչները: Համեմատի՛ր այդ որոշիչները:
17. Ներմուծի՛ր n թիվ, *numpy* գրադարանով հաշվի՛ր և տպի՛ր e^n -ը:
18. Ստեղծի՛ր 3×3 չափանի մատրից՝ [0, 20] միջակայքի պատահական թվերով: Տպի՛ր այդ մատրիցի բոլոր տարրերի գումարը:

19. Ստեղծի՛ր 4×4 չափանի մատրից՝ $[0, 50]$ միջակայքի պատահական թվերով: Հաշվի՛ր և տպի՛ր բոլոր այն թվերի քանակը, որոնք մեծ են մատրիցի բոլոր թվերի միջինից:
20. Ստեղծի՛ր 4×4 չափանի մատրից՝ $[0, 50]$ միջակայքի պատահական թվերով: Գլխավոր անկյունագծի տարրերը դարձրո՛ւ 0: Տպի՛ր ստացված մատրիցը:

Լուծում

Python

```
import numpy as np
```

```
# Ստեղծում ենք 4x4 չափի պատահական ամբողջ թվերով մատրից 0-100 միջակայքում
matrix = np.random.randint(0, 50, (4, 4))
```

```
# Տպում ենք նախնական մատրիցը
print("Նախնական մատրիցա:\n", matrix)
```

```
for i in range(4):
```

```
    matrix[i,i] = 0
```

```
# Տպում ենք ստացված մատրիցը
```

```
print("Նոր մատրիցը", matrix)
```

21. Ստեղծի՛ր 4×4 չափի մատրից՝ $[0, 50]$ միջակայքի պատահական թվերով:
Գլխավոր անկյունագծից ներքև բոլոր տարրերը դարձրո՛ւ 1: Տպի՛ր ստացված մատրիցը:
22. Ստեղծի՛ր 4×4 չափի մատրից՝ $[0, 50]$ միջակայքի պատահական թվերով: Հաշվի՛ր գլխավոր անկյունագծից վերև գտնվող բոլոր տարրերի գումարը:
23. Ստեղծի՛ր 4×4 չափի պատահական ամբողջ թվերով գեներացվող սիմետրիկ մատրից:
 M մատրիցը կոչվում է սիմետրիկ, եթե $M[i, j] = M[j, i]$, բոլոր i, j -երի համար:
24. Ստեղծի՛ր 4×4 չափի մատրից՝ $[0, 20]$ միջակայքի պատահական թվերով: Նորմալացրո՛ւ մատրիցը $\min - \max$ -ի եղանակով և տպի՛ր այն:
 $M[i, j] = \frac{A[i, j] - \min(A)}{\max(A) - \min(A)}$, որտեղ $M[i, j]$ -ն նոր մատրիցի համապատասխան տարրն է, $A[i, j]$ -ն սկզբնական մատրիցը, $\min(A)$ -ն և $\max(A)$ -ն համապատասխանաբար A -ի փոքրագույն և մեծագույն արժեքները:
Մատրիցի նորմալացումը մաթեմատիկական մի գործողություն է, որի արդյունքում մատրիցի բոլոր տարրերը համապատասխանեցվում են որոշակի միջակայքի՝ սովորաբար $[0, 1]$:

MATPLOTLIB PYPLOT ԳՐԱՂԱՐԱՆ

Matplotlib գրադարանը ստեղծվել է 2003թ. կենսաբժշկական ճարտարագիտության ոլորտի մասնագետ Ջոն Դ. Հանթերի կողմից: Ջոն Հանթերը Python լեզվով իրականացնում էր բժշկական հետազոտությունների նախագծեր և կարիք ուներ՝ պարզ և ճկուն կերպով պատկերելու տվյալները:



Ջոն Հանթեր

Այդ տարիներին Python-ում գրեթե չկային հարմար պատկերավորման գրադարաններ, իսկ MATLAB ծրագիրը վճարովի էր և չունեւ բաց կոդ:

Հանրերը նպատակ ունեւ ստեղծել բաց կոդով (open-source) կայուն պատկերավորման համակարգ, որը թույլ կտար Python-ում ստանալ նույն հնարավորությունները, ինչ MATLAB-ում: 2003 թ. Matplotlib-ի առաջին տարբերակը հրապարակվեց որպես պարզ գործիքակազմ՝ 2D գրաֆիկներ կառուցելու համար: Matplotlib-ն արագորեն դարձավ Python-ի գիտական Էկոհամակարգի առանցքային մաս: Այսօր Matplotlib-ի զարգացումն իրականացնում է Python-ի մեծ համայնքը և Matplotlib Development Team-ը:

Matplotlib-ի առավելությունները

- Հին և կայուն գրադարան է, մեծ համայնքով և լավ փաստաթղթավորված է:
- Բազմակողմանիություն՝ կարող է ստեղծել գրեթե ցանկացած տեսակի գրաֆիկ:
- Հասանելիություն՝ հեշտ է տեղադրել, անվճար է և բաց կոդով:
- Պլատֆորմային անկախություն՝ աշխատում է Windows, macOS, Linux ՕՏ-երում:
- Համատեղելիություն՝ աշխատում է Pandas, NumPy, Jupyter Notebook, Seaborn և այլ գործիքների հետ:
- Կարելի է արտահանել պատկերներ PNG, PDF, SVG, EPS և այլ ֆորմատներով:
- Հարմար է հատուկ և ճկուն կարգավորումների համար:

Տարիներ	Հիմնական իրադարձություն	Նորամուծություններ և նշանակություն
2003	Նախագծի ստեղծում Հանրերի կողմից:	Ստեղծվեց որպես MATLAB-ին բաց կոդով այլընտրանք:
2005–2007 (v0.90+)	Համայնքի կայունացում: աճ,	Object-Oriented API-ի առաջին տարբերակը, ինտեգրում NumPy/SciPy-ի հետ:
2008–2010	Մասշտաբային զարգացում:	Figure-Axes-Artist կառուցվածքի ձևավորում, առաջատար պատկերավորման գործիք Python-ում:
2010–2012 (v1.0)	Լայն տարածում և ընդունում:	Matplotlib-ը դարձավ տվյալագիտության և գիտության ստանդարտ պատկերավորման գրադարանը:
2012	Հանրերի մահը:	Նախագիծն անցավ Matplotlib Development Team-ին:

2014–2017 (v1.5 → v2.0)	Մոդեռնիզացիա:	Նոր լռելյայն ոճ, բարելավված գունային սխեմաներ և դիզայն:
2018–2020 (v3.0+)	Ինտեգրում Python3-ում:	Python 3-ի ամբողջական աջակցություն, նոր միջերեսներ (interface) և ipywidgets-ի ինտեգրում:
2020–մինչ օրս (v3.3+)	Ժամանակակից փուլ:	Արագ տեսապատկերում, հեշտացված API, նոր պատկերվորման հնարավորություններ, ակտիվ զարգացում:

Matplotlib-ի կիրառություններ

- **NASA, SpaceX** և այլ ինժեներական ընկերություններ օգտագործում են տիեզերական տվյալների վիզուալիզացիայի համար:
- **CERN**՝ ֆիզիկայի հետազոտությունների արդյունքների ցուցադրման համար:
- **Google** և **Microsoft**՝ տվյալագիտության և AI նախագծերում:
- **Netflix, Uber, Airbnb**՝ տվյալների վերլուծության և բիզնես վիզուալիզացիայի համար:
- **Ակադեմիական ոլորտ**՝ համալսարաններում ու գիտական հոդվածներում մեծամասամբ օգտագործում են Matplotlib-ի գրաֆիկները:

Հիմնական հրահանգներ

- `import matplotlib.pyplot as plt` ներմուծում է *Matplotlib*-ի *pyplot* ենթամոդուլը, որը հիմնականում օգտագործում են՝ բոլոր տեսակի գրաֆիկներ ստեղծելու համար:
- `plt.show()`՝ ցուցադրում է գրաֆիկն էկրանին. անհրաժեշտ է յուրաքանչյուր գրաֆիկի վերջում, որպեսզի այն տեսանելի լինի:

ԳԾԱՅԻՆ ԳՐԱՖԻԿ (LINE PLOT)

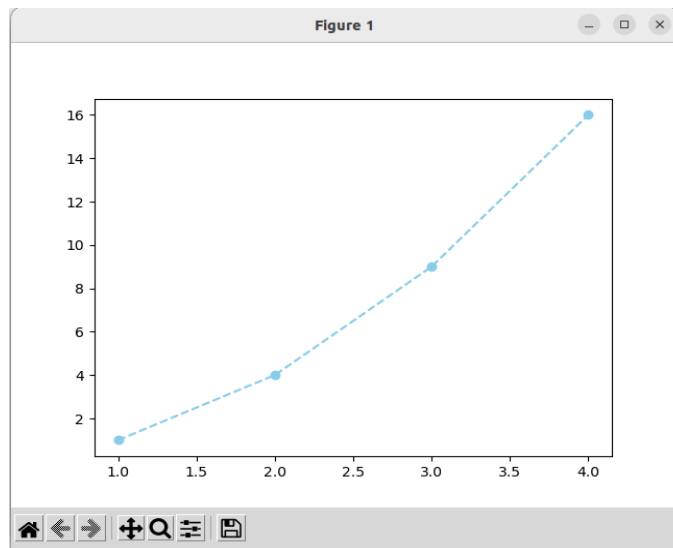
Սահմանում: Գծային դիագրամը ներկայացվում է (x,y) կետերի և այդ կետերը հաջորդաբար միացնող ուղիղ հատվածներով: Այն հատկապես հարմար է՝ ժամանակային շարքերը ներկայացնելու համար, որտեղ x-առանցքը ցույց է տալիս ժամանակը:

- `plt.plot(x, y)`՝ ստեղծում է գծային գրաֆիկ՝ օգտագործելով *x* և *y* տվյալները:
- Հատկանիշներ՝ *marker* (կետերի տեսքը), *color* (գույնը), *linestyle* (գծի տեսքը), *ms* (կետի չափը): Ավելի մանրամասն կարող եք տեսնել այստեղ: Օրինակ՝

Python

```
import matplotlib.pyplot as plt

# Գծային գրաֆիկի օրինակ
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]
plt.plot(x, y, color='skyblue', linestyle='--', marker='o')
plt.show()
```



Նույն առանցքի վրա կարելի է ստեղծել 2 գծային գրաֆիկ՝

Python

```
import matplotlib.pyplot as plt

# Տվյալներ
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
temperature = [3, 5, 9, 13, 18, 21, 25, 24, 20, 14, 8, 4] # ջերմաստիճան (°C)
precipitation = [78, 54, 62, 48, 71, 67, 58, 76, 82, 98, 89, 77] # տեղումների մակարդակ (mm)

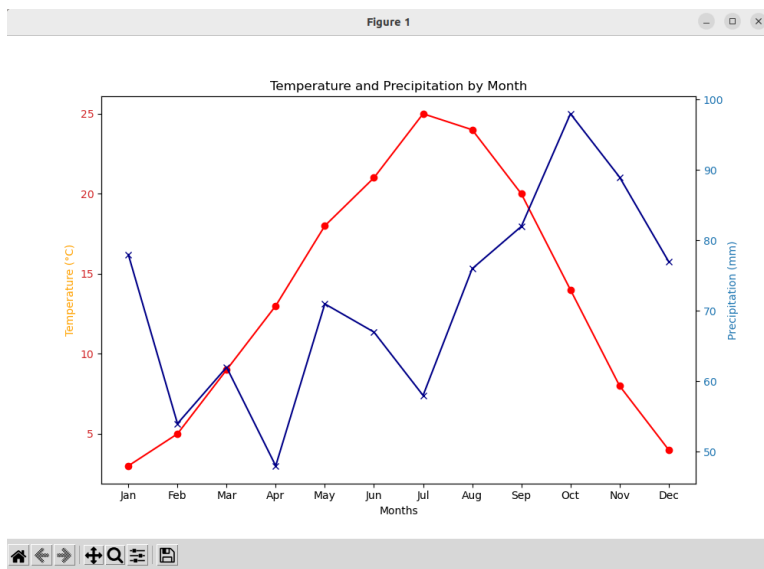
# Ստեղծում ենք գծապատկեր՝ առաջին y-առանցքով
fig, ax1 = plt.subplots(figsize=(10, 6))

# Գծում ենք ջերմաստիճանը
ax1.plot(months, temperature, color="red", marker="o", label="Temperature (°C)")
ax1.set_xlabel("Months")
ax1.set_ylabel("Temperature (°C)", color="orange")
ax1.tick_params(axis="y", labelcolor="tab:red")
```

```
# Ստեղծում ենք երկրորդ y-առանցքը նույն x-առանցքի համար
ax2 = ax1.twinx()
ax2.plot(months, precipitation, color="darkblue", marker="x", label="Precipitation (mm)")
ax2.set_ylabel("Precipitation (mm)", color="tab:blue")
ax2.tick_params(axis="y", labelcolor="tab:blue")

# Վերնագիր
plt.title("Temperature and Precipitation by Month")

# Ցույց ենք տալիս գծապատկերը
plt.show()
```



ՍՅՈՒՆԱԿԱՁԵՎ ԴԻԱԳՐԱՄ (Bar Plot)

Սահմանում: Սյունյակաձև դիագրամը տվյալները ներկայացնում է ուղղանկյուն սյուներով (բարերով), որոնց երկարությունը կամ բարձրությունը համեմատական է դրանց ներկայացնող արժեքներին: Սյուները կարող են տեղադրվել ուղղահայաց կամ հորիզոնական:

- `plt.bar(x, height)`՝ ստեղծում է ուղղահայաց սյունակային գրաֆիկ՝ օգտագործելով x արժեքները և համապատասխան բարձրությունները:
 - `plt.barh(y, width)`՝ ստեղծում է հորիզոնական սյունակային գրաֆիկ՝ օգտագործելով x արժեքները և համապատասխան բարձրությունները:
- Հատկանիշներ՝ `color` (գույն), `width` (հաստություն): Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
months = ['Յունիս', 'Փետրվար', 'Մարտ', 'Ապրիլ', 'Մայիս']
```

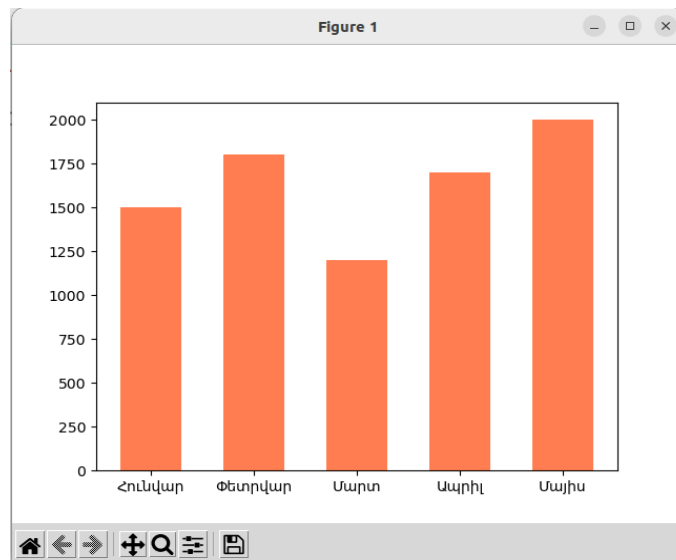
```
sales = [1500, 1800, 1200, 1700, 2000]
```

```
# Սյունակային գրաֆիկի կառուցում
```

```
plt.bar(months, sales, color='#FF7F50', width=0.6)
```

```
# Գրաֆիկի ցուցադրում
```

```
plt.show()
```



Python

```
import matplotlib.pyplot as plt
```

```
# Տվյալներ
```

```
months = ['Հունվար', 'Փետրվար', 'Մարտ', 'Ապրիլ', 'Մայիս']
```

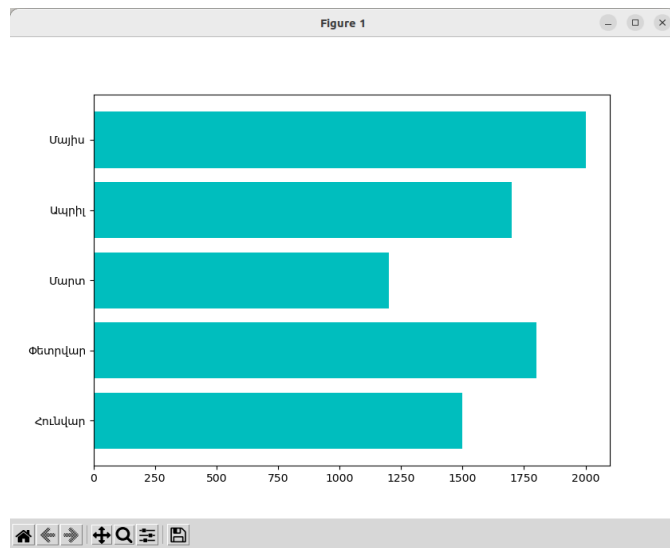
```
sales = [1500, 1800, 1200, 1700, 2000]
```

```
# Սյունակային գրաֆիկի կառուցում
```

```
plt.barh(months, sales, color='c')
```

```
# Գրաֆիկի ցուցադրում
```

```
plt.show()
```



ՏԱՐԱԾՔԱՅԻՆ ԳՐԱՖԻԿ (Area Plot)

Ասահմանում: Տարածքային դիագրամը գծային դիագրամի տարատեսակ է, որտեղ գծի տակ գտնվող տարածքը լցված է գույնով: Երբեմն մի քանի հաստատունների արժեքները դրվում են մեկը մյուսի վրա՝ ամբողջական պատկեր ստանալու համար:

- `plt.fill_between(x, y1, y2)`՝ նկարագրում է երկու գծերի միջև գտնվող տարածքը՝ օգտագործելով `x` արժեքները և `y1`, `y2` սահմանները: Հատկանիշներ՝ `color` (գույն), `alpha` (թափանցիկություն):

Օրինակներ՝

Python

```
import matplotlib.pyplot as plt
```

```
# Տվյալներ
```

```
days = [1, 2, 3, 4, 5]
```

```
temperature_city1 = [20, 22, 23, 24, 26]
```

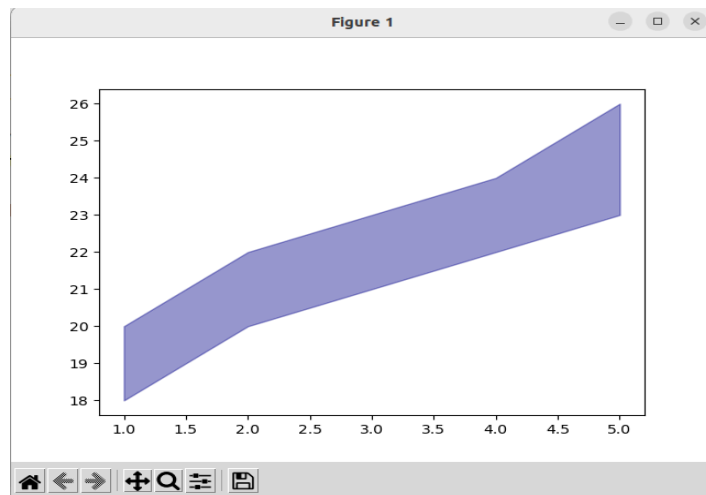
```
temperature_city2 = [18, 20, 21, 22, 23]
```

```
# Տարածքային գրաֆիկի կառուցում
```

```
plt.fill_between(days, temperature_city1, temperature_city2, color='darkblue',  
alpha=0.4)
```

```
# Գրաֆիկի ցուցադրում
```

```
plt.show()
```



Python

```
import matplotlib.pyplot as plt
```

Տվյալներ

```
days = [1, 2, 3, 4, 5]
```

```
temperature_city1 = [20, 21, 23, 24, 26]
```

```
temperature_city2 = [18, 24, 21, 20, 19]
```

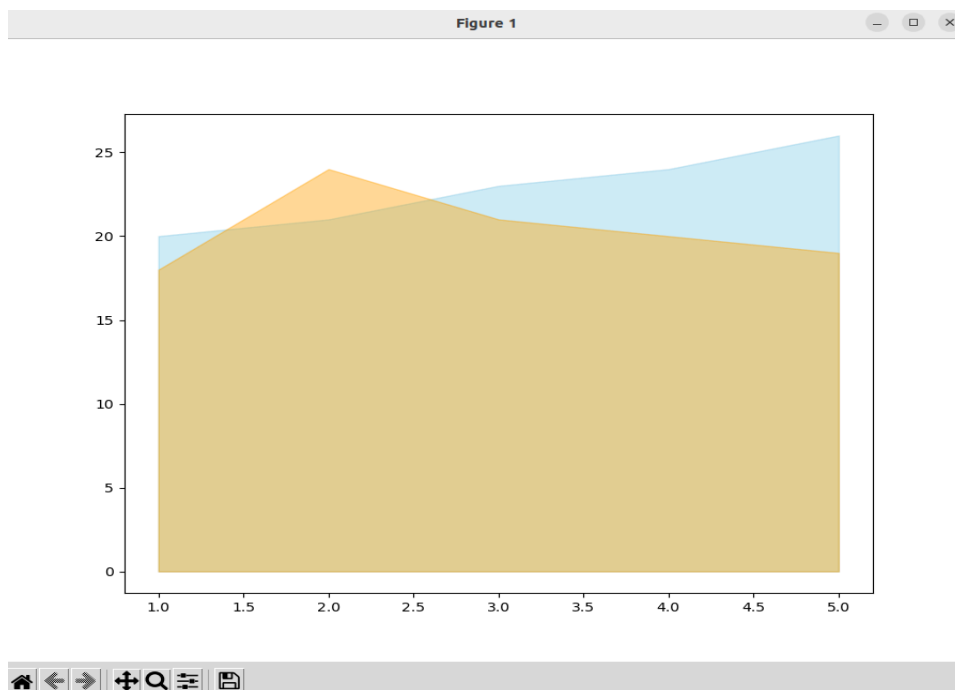
Տարածքային գրաֆիկի կառուցում

```
plt.fill_between(days, temperature_city1, color='skyblue', alpha=0.4, label='Քաղաք 1')
```

```
plt.fill_between(days, temperature_city2, color='orange', alpha=0.4, label='Քաղաք 2')
```

Գրաֆիկի ցուցադրում

```
plt.show()
```



ՑՐԱԿԵ ԳՐԱՖԻԿ (Scatter Plot)

Սահմանում: Ցրված գրաֆիկը տվյալների (x,y) կետեր է ցուցադրում Դեկարտյան կոորդինատային համակարգում՝ երկու թվային փոփոխականների արժեքները ներկայացնելու համար:

- `plt.scatter(x, y)`՝ ստեղծում է ցրված գրաֆիկ՝ օգտագործելով *x* և *y* տվյալները: Հատկանիշներ՝ *color* (գույն), *marker* (կետի տեսք), *s* (կետի չափ): Օրինակ՝

Python

```
import matplotlib.pyplot as plt
```

Տվյալներ

```
math = [55, 62, 13, 44, 65, 60, 78, 81, 45, 91, 32, 65, 87, 85, 54, 76]
```

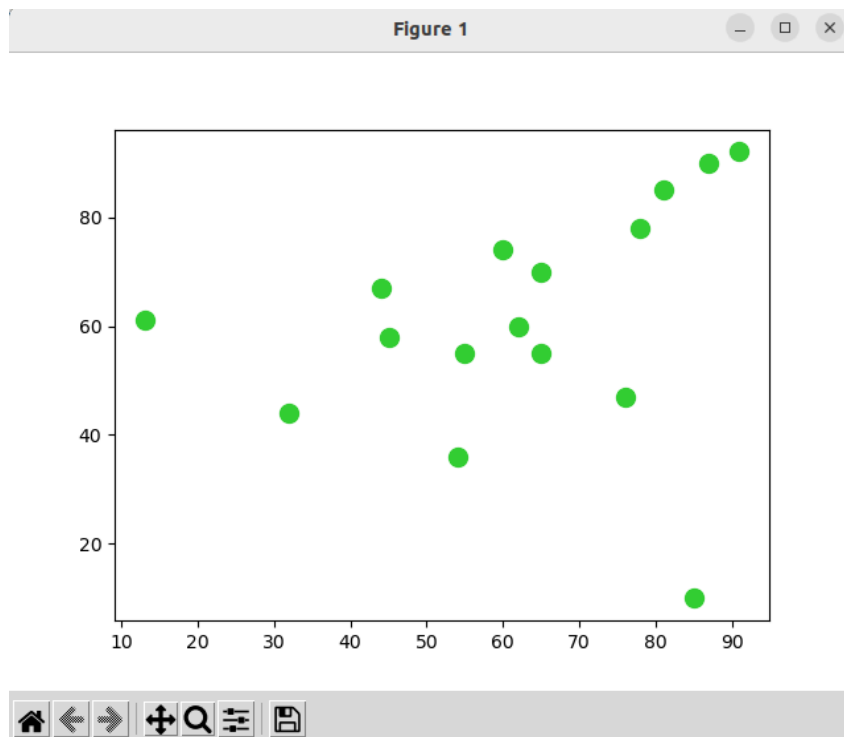
```
python = [55, 60, 61, 67, 70, 74, 78, 85, 58, 92, 44, 55, 90, 10, 36, 47]
```

Ցրված գրաֆիկի կառուցում

```
plt.scatter(math, python, color='limegreen', marker='o', s=100)
```

Գրաֆիկի ցուցադրում

```
plt.show()
```



ՀԻՍՏՈԳՐԱՄ (Histogram)

Սահմանում: Հիստոգրամը տվյալների բաշխումը պատկերող գրաֆիկական դիագրամ է, որը

ցույց է տալիս, թե թվային տվյալների արժեքներն ինչ հաճախականությամբ են հանդիպում որոշակի միջակայքերում:

- `plt.hist(data, bins = n)`՝ կառուցում է հիստոգրամ:
Հատկանիշներ՝ `color` (գույն), `edgecolor` (եզրագծի գույն), `alpha` (թափանցիկություն):

Օրինակ՝

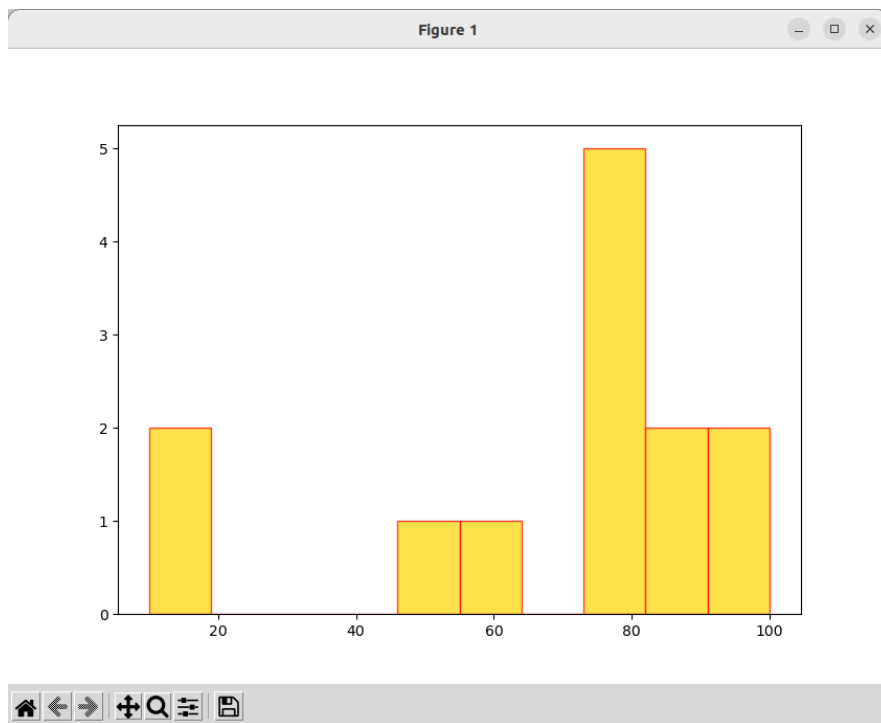
Python

```
import matplotlib.pyplot as plt

# Տվյալներ (օրինակ՝ 1000 պատահական թիվ)
data = [ 100, 10, 80, 78, 90, 50, 78, 14, 56, 80, 99, 78, 88]

# Հիստոգրամի կառուցում
plt.hist(data, bins=10, color='gold', edgecolor='red', alpha=0.7)

# Գրաֆիկի ցուցադրում
plt.show()
```



ՇՐՋԱՆԱՅԻՆ ԴԻԱԳՐԱՄ (Pie Chart)

Սահմանում: Շրջանային դիագրամը շրջան է, որը բաժանված է հատվածների (սեկտորների):

Յուրաքանչյուր հատվածի մակերեսը ներկայացնում է տվյալների բաշխման մեջ դրա մասնաբաժինը (տոկոսը):

- `plt.pie(sizes, labels = labels)` ստեղծում է շրջանաձև դիագրամ:

Հատկանիշներ՝ `colors` (գույն), `explode` (կտորների հեռավորություն), `autopct` (տոկոսների ցուցադրում), `startangle` (մեկնարկային անկյուն):

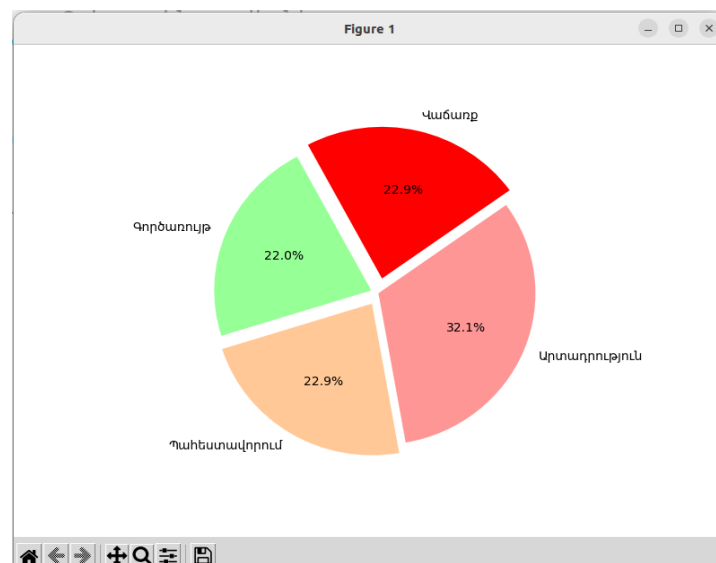
Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
chart_labels = ['Արտադրություն', 'Վաճառք', 'Գործառույթ', 'Պահեստավորում']
sizes = [35, 25, 24, 25] # Տոկոսային արժեքներ
chart_colors = ['#FF9999', 'red', '#99FF99', '#FFCC99'] # Գույներ յուրաքանչյուր հատվածի համար
chart_explode = (0, 0.1, 0.05, 0.08) # Որոշ հատվածների առանձնացում

# Կտորային գրաֆիկի կառուցում
plt.pie(sizes, labels=chart_labels, colors=chart_colors, explode=chart_explode, autopct='%1.1f%%', startangle=280)

# Գրաֆիկի ցուցադրում
plt.show()
```



ԴՆԴՁԱԿԱՅԻՆ ԴԻԱԳՐԱՄ (Bubble Chart)

Սահմանում: Այս դիագրամը ցրված գրաֆիկի տեսակ է, որում յուրաքանչյուր արժեք ներկայացված է

ոչ թե կետով, այլ պղպջակով (շրջանակով), և այդ պղպջակի չափը պատկերում է տվյալների երրորդ չափը (փոփոխականը):

- `plt.scatter(x, y, s = size)`՝ ցրված գրաֆիկի տեսակ է, որտեղ յուրաքանչյուր կետ (բաղադրիչ) ներկայացվում է որպես պղպջակ: Պղպջակի դիրքը (X և Y արժեքները) որոշվում է X , Y տվյալների երկու փոփոխականներով, իսկ պղպջակի չափը՝ երրորդ փոփոխականով:
Հատկանիշներ՝ `color` (գույն), `alpha` (թափանցիկություն), `cmap` (գրադիենտային գունավորում, որն ունի '`viridis`', '`plasma`', '`inferno`', '`coolwarm`' և այլ ոճեր):

Օրինակ՝

Python

```
import matplotlib.pyplot as plt
```

```
# Տվյալներ
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 15, 30, 25]
```

```
size = [100, 222, 500, 400, 350] # Պղպջակի չափը
```

```
colors = [1, 2, 3, 4, 5] # Պղպջակների գույնը
```

```
# Bubble Chart-ի կառուցում՝ գրադիենտով գույնով
```

```
plt.scatter(x, y, s=size, c=colors, cmap='plasma', alpha=0.6)
```

```
# Վերնագիր և առանցքային պիտակներ
```

```
plt.title("Պղպջակային դիագրամ գրադիենտային գունավորմամբ")
```

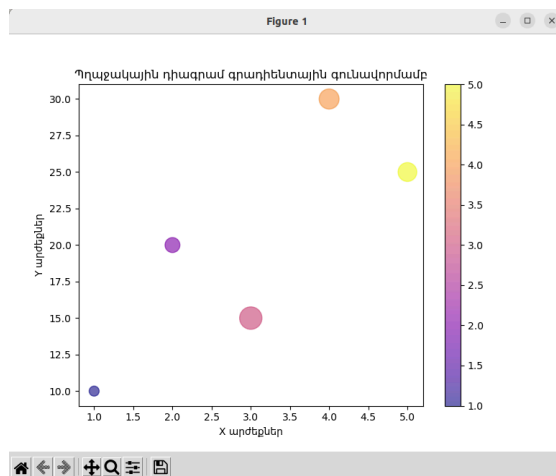
```
plt.xlabel("X արժեքներ")
```

```
plt.ylabel("Y արժեքներ")
```

```
# Գրաֆիկի ցուցադրում
```

```
plt.colorbar() # Ավելացնել գույնի սանդղակ
```

```
plt.show()
```



ՏՈՒՓԱՅԻՆ (ՈՒՂՂԱՆԿՅՈՒՆԱՁԵՒ) ԴԻԱԳՐԱՄ (Box Plot)

Սահմանում: Տուփային դիագրամը (Boxplot) օգտագործվում է՝ տվյալների բաշխվածությունը, կենտրոնական միտումները և ծայրահեղ արժեքներն արագ ու հստակ ցույց տալու համար:

- `plt.boxplot(data)`՝ ստեղծում է տուփային դիագրամ՝ ըստ տրված տվյալների:

Օրինակ՝

Python

```
import matplotlib.pyplot as plt
```

```
# Նորածինների քաշերը կգ-երով
```

```
newborn_weights = [3.2, 3.5, 2.8, 3.1, 4.0, 2.9, 3.3, 3.6, 2.7, 3.0, 3.4, 3.1, 3.8, 2.6, 3.9, 3.2, 3.5, 3.0, 2.5, 3.7]
```

```
# Տուփի դիագրամի կառուցում
```

```
plt.figure(figsize=(8, 6)) # չափեր
```

```
plt.boxplot(newborn_weights, vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue'))
```

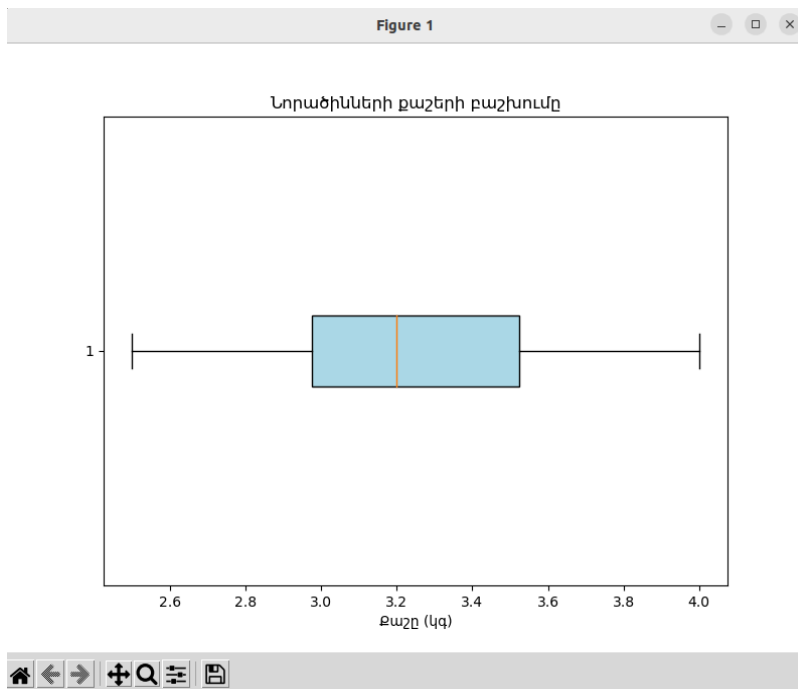
```
# Ավելացնում ենք վերնագիր և պիտակներ
```

```
plt.title("Նորածինների քաշերի բաշխումը")
```

```
plt.xlabel("Քաշը (կգ)")
```

```
# Ցույց ենք տալիս դիագրամը
```

```
plt.show()
```



Box plot կարելի է նաև կառուցել, եթե ունենք մեկից ավելի տվյալների խմբեր:

```
Python
import matplotlib.pyplot as plt

# Նորածինների քաշերը կգ-երով
newborn_boys = [3.2, 3.5, 2.8, 3.1, 4.0, 2.9, 3.3, 3.6, 2.7, 3.0,]
newborn_girls = [3.4, 3.1, 3.8, 2.6, 3.9, 3.2, 3.5, 3.0, 2.5, 3.7, 2.8]

# Տուփի դիագրամի կառուցում
plt.figure(figsize=(8, 6)) # չափեր
plt.boxplot([newborn_girls, newborn_boys], labels=['Աղջիկ', 'Տղա'], vert=False, patch_artist=True,
            boxprops=dict(facecolor='lightblue'))

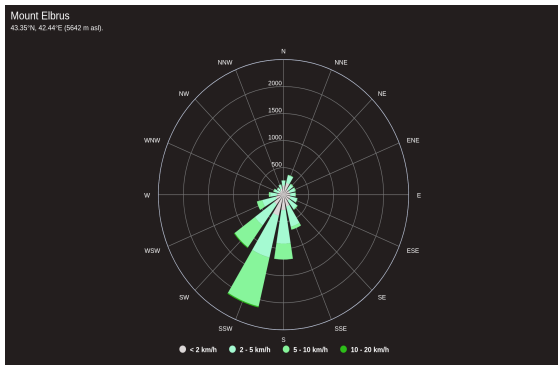
# Ավելացնում ենք վերնագիր և պիտակներ
plt.title("Նորածինների քաշերի բաշխումը")
plt.xlabel("Քաշը (կգ)")

# Ցույց ենք տալիս դիագրամը
plt.show()
```

Տուփ (*Box*)-ը ցույց է տալիս տվյալների երկրորդ և երրորդ քառորդների (25% – 75%) միջակայքը: Տուփի ներսում գիծ է, որը ցույց է տալիս տվյալների կենտրոնական արժեքը՝ մեդիանը: Տուփից ձգվում են գծեր, որոնք ընդգրկում են հիմնական բաշխումը՝ բաց թողնելով եզրային արժեքները, իսկ եզրային արժեքները տուփի և ուսածողերի սահմաններից դուրս գտնվող կետերն են, որոնք տարբերվում են տվյալների ընդհանուր բաշխումից:

ԲԵՎԵՌԱՅԻՆ ԳՐԱՖԻԿ (Polar plot)

Սահմանում: Բևեռային գրաֆիկը տվյալների տեսապատկերման տեսակ է, որտեղ տվյալները ներկայացվում են բևեռային կոորդինատների համակարգում: Այսինքն՝ յուրաքանչյուր կետ սահմանվում է ոչ թե (x, y) ուղղանկյուն կոորդինատներով, այլ (θ, r) կետերով, որտեղ θ -ն անկյան չափն է, իսկ r -ը՝ շառավիղ: Բևեռային գրաֆիկով պատկերում են քամու ուղղությունները և արագությունները, այն հայտնի է քամու վարդ անունով (wind rose):



Էլբրուս լեռան վրա բամուռ ուղղությունները և արագությունները պատկերող բևեռային գրաֆիկ: Նկարը` <https://www.meteoblue.com/> կայքի

«Բամուռ վարդն» օգտագործում են խոր բնակավայրերի, շենքերի կառուցման ժամանակ:

Այս գրաֆիկը ստանանք matplotlib-ով`

```
Python
import numpy as np
import matplotlib.pyplot as plt

# Ուղղություններ
angles = np.linspace(0, 2*np.pi, 16, endpoint=False)

# Տվյալներ
values = [200, 250, 300, 200, 150, 100, 150, 190,
          250, 600, 1250, 2200, 1200, 600, 300, 250]

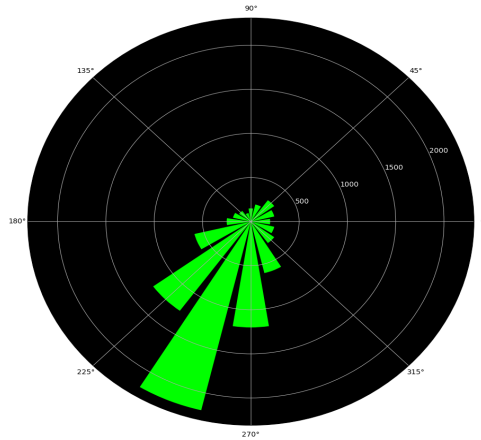
# Բևեռային առանցք
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'}, figsize=(8, 8))
ax.set_facecolor("black")

# Գծեր
ax.bar(angles, values, width=np.pi/10, color="lime")

# Շրջանաձև նշումներ
ax.set_yticks([500, 1000, 1500, 2000])
ax.set_yticklabels(['500', '1000', '1500', '2000'], color="white")

# Վերնագիր
plt.title("Mount Elbrus\n43.35°N, 42.44°E (5642 m).",
          color="white", loc="left")

plt.show()
```



Իհարկե, կարելի է գրաֆիկն ավելի մանրամասն ստանալ, սակայն սա պարզ օրինակն է:

Բևեռային գրաֆիկի ավելի պարզ տեսքն այսպիսինն է՝

Python

```
import matplotlib.pyplot as plt
import numpy as np
```

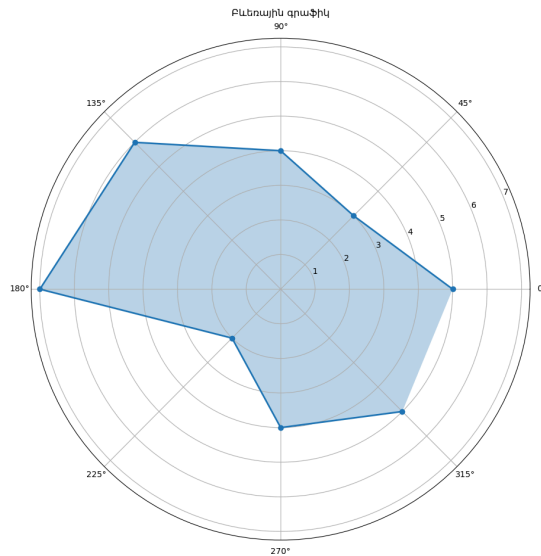
```
# Անկյունների ցուցակ (աստիճաններով, որը փոխում ենք ռադիանների)
angles_deg = [0, 45, 90, 135, 180, 225, 270, 315]
theta = [np.deg2rad(a) for a in angles_deg] # աստիճանից ռադիան փոփոխում
```

```
# Շառավղի արժեքները
r = [5, 3, 4, 6, 7, 2, 4, 5]
```

```
# Գծում
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
ax.plot(theta, r, marker='o', linewidth=2)
ax.fill(theta, r, alpha=0.3)
```

```
# Տիտղոս
ax.set_title("Բևեռային գրաֆիկ", va='bottom')
```

```
plt.show()
```



Գրաֆիկի ձևավորում

- `plt.title("Գրաֆիկի տիտղոս")`՝ սահմանում է գրաֆիկի վերնագիրը:
- `plt.xlabel("X առանցքի անուն")`՝ սահմանում է X առանցքի անունը:
- `plt.ylabel("Y առանցքի անուն")`՝ սահմանում է Y առանցքի անունը:
- `plt.legend()`՝ գրաֆիկին ավելացնում է պիտակ (*label*)՝ յուրաքանչյուր գույնի կամ գծի նշանակությունը ցուցադրելու համար:
- `plt.style.use("style name")`՝ այս ֆունկցիան թույլ է տալիս կիրառել նախապես սահմանված ոճեր՝ գրաֆիկների արտաքին տեսքը ձևափոխելու համար: `matplotlib`-ը պարունակում է մի քանի ներկառուցված ոճեր, ինչպիսին են՝ *ggplot*, *seaborn*, *bmh* և այլն:

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Ընտրում ենք ոճը , այս ոճը ունի իր գույները և այլ գույներ սահմանել պետք է
plt.style.use('tableau-colorblind10')

# Տվյալներ
```

```

x = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [1, 3, 6, 10, 15]

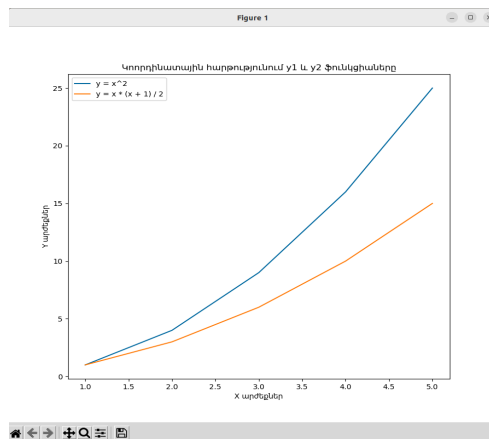
# գծերի գծագրում
plt.plot(x, y1, label="y = x^2")
plt.plot(x, y2, label="y = x * (x + 1) / 2")

# Պիտակի ցուցադրում
plt.legend()

# Գրաֆիկի ցուցադրում
plt.title("Կոորդինատային հարթությունում y1 և y2 ֆունկցիաները")
plt.xlabel("X արժեքներ")
plt.ylabel("Y արժեքներ")

plt.show()

```



ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ստեղծի՛ր գծային գրաֆիկ՝ $y = 3x$ գծի համար: Գրաֆիկի համար x -ին տո՛ւր հետևյալ արժեքները՝ 0, 1, 2, 3:
2. Ստեղծի՛ր հետևյալ ֆունկցիայի $y = x^2$ գրաֆիկը, x -ին արժեքները տո՛ւր – 5 -ից մինչև 5՝ ընտրելով ամբողջ թվերը:

3. Ստեղծի՛ր գծային գրաֆիկ՝ ներկայացնելու ջերմաստիճանի փոփոխությունը ժամը 8:00-ից մինչև 20:00-ը՝ ըստ հետևյալ տվյալների՝

Python

Ժամեր = [8, 10, 12, 14, 16, 18, 20]

Ցերմաստիճան (°C) = [10, 15, 20, 22, 21, 18, 16]

4. Կառուցի՛ր սյունակաձև դիագրամ՝ ՀՀ ամենաշատ բնակեցված քաղաքների բնակչության տվյալներով՝

Python

Քաղաքներ = ['Երևան', 'Գյումրի', 'Վանաձոր', 'Ջրագորան', 'Աբովյան', 'Կապան']

Բնակչություն = [1000000, 120000, 80000, 40000, 35000, 30000]

5. Կառուցի՛ր հիստոգրամ՝ ցույց տալու ուսանողների գնահատականների բաշխումը հետևյալ տվյալներով՝

Python

Գնահատականներ = [55, 78, 67, 90, 83, 45, 88, 90, 79, 65, 84, 92, 70]

6. Կառուցի՛ր հիստոգրամ, որը ցույց կտա Ասիա մայրցամաքի տրված լեռների բարձրությունների բաշխումը՝

Python

Լեռների բարձրություն (մ) = [8848, 8611, 8586, 8516, 8463, 8201, 8167, 8125, 8091, 8078, 8051, 8010, 7925, 7825, 7788, 7690, 7555, 7546, 7526, 7500]

7. Հիստոգրամով ներկայացրո՛ւ տարբեր տարիների վազքի մրցույթների հաղթողների տարիքը՝

Python

Տարիքներ = [25, 32, 30, 38, 30, 35, 35, 28, 25, 32, 26, 21, 29, 24, 28, 19]

8. Կառուցի՛ր շրջանային դիագրամ՝ ցուցադրելով արշավական տարբեր պարագաների տեսակների վաճառքի տոկոսային բաժինը՝

Python

```
Պարագաներ = [ 'Կոշիկ', 'Ուսապարկ', 'Թերմոս', 'Քնապարկ', 'Վրան', ]  
Վաճառք (հազ. դրամ) = [400, 200, 30, 240, 270]
```

9. Կառուցի՛ր շրջանային դիագրամ՝ ցուցադրելով քննության արդյունքների տոկոսային բաժինը՝

Python

```
Պարագաներ = ['Անբավարար', 'Բավարար', 'Լավ', 'Գերազանց', ]  
Գնահատականներ = [5, 7, 3, 7, 8, 6, 4, 7, 3, 6, 10, 4, 5]
```

10. Կառուցի՛ր տարածքային գրաֆիկ (*Area Plot*)՝ ներկայացնելով եկամուտներն ու ծախսերը հետևյալ ամիսների համար՝

Python

```
Ամիսներ = ['Հունվար', 'Փետրվար', 'Մարտ', 'Ապրիլ']  
Եկամուտներ ($) = [2000, 2200, 1800, 2100]  
Ծախսեր ($) = [1500, 1600, 1300, 1500]
```

11. Կազմի՛ր տարածքային գրաֆիկ (*Area Plot*), որը ցույց կտա Երևանի, Գյումրիի և Վանաձորի միջին ջերմաստիճանները մի քանի ամիսների ընթացքում, հետևյալ տվյալներով՝

Python

```
Ամիսներ = ['Սեպտեմբեր', 'Հոկտեմբեր', 'Նոյեմբեր', 'Դեկտեմբեր', 'Հունվար', 'Փետրվար']  
Երևան=[23, 15, 8, 3, -1, 2]  
Գյումրի=[20, 13, 6, 0, -5, -2]  
Վանաձոր=[22, 14, 7, 2, -3, 0]
```

12. Կազմի՛ր ցրված գրաֆիկ (*scatter plot*)՝ ցուցադրելով մարդկանց հասակի և քաշի տվյալները՝

Python

```
Հասակ (սմ) = [160, 165, 170, 175, 180, 185, 163]  
Քաշ (կգ) = [50, 55, 63, 70, 75, 85, 77]
```

13. Կազմի՛ր ցրված գրաֆիկ՝ ցուցադրելով աշակերտների գնահատականների տվյալները՝

Python

Հանրահաշիվ = [25, 78, 45, 90, 95, 55, 16, 67, 24, 100, 32, 65]

Անգլերեն = [36, 84, 80, 92, 65, 45, 48, 52, 90, 76, 39, 51]

14. Կազմի՛ր պղպջակային դիագրամ (*bubble chart*), որտեղ x -ը կլինի ծովի մակարդակից բարձրությունը, y -ը՝ օդի խտությունը, իսկ պղպջակի չափը կբնութագրի արևի վտանգավոր ճառագայթների ինտենսիվությունը:

ԲՇՄ, մ	Օդի խտությունը գ/մ ³	Արևի վտանգավոր ճառագայթների ինտենսիվության չափը
1500	1225	1050
2000	1210	1100
2500	1200	1120
3000	1180	1170
3500	1155	1250
4000	1120	1350

15. Կազմի՛ր պղպջակային դիագրամ, որտեղ x -ը կլինի քաղաքի կանաչ տարածքի մակերեսը, y -ը՝ ամռան ամսիկների միջին ջերմաստիճանը, իսկ պղպջակի չափը կբնութագրի կանաչ տարածքների պահպանման ծախսը:

Կանաչ տարածքի մասնաբաժինը, %	Ամառվա միջին ջերմաստիճան, °C	Ծախս (հազար \$)
25	37	105
45	30	90
60	32	150
75	25	85

16. Ունենք Նեպալի Նամչե Բազար գյուղի բնակիչների հասակների տվյալները: Կազմի՛ր տուփային դիագրամ (*box plot*) նրանց հասակների տվյալներով՝

Python

height=[150,165, 170, 155, 180, 160, 158, 172, 164, 210, 175, 169, 175, 178, 167, 163, 159, 161, 173,198]

Քանի՞ եզրային արժեք կա:

17. Առկա են ուսանողների գնահատականները հանրահաշիվ, անգլերեն և ֆիզիկա առարկաներից: Կազմի՛ր տուփային դիագրամ (*box plot*) այդ տվյալներով՝

Python

Հանրահաշիվ = [20, 75, 85, 90, 78, 92, 88, 24, 76, 83, 44, 95, 81, 0]

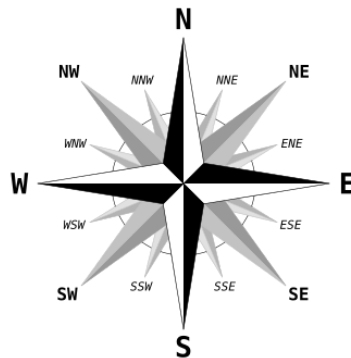
Անգլերեն = [70, 80, 65, 88, 55, 91, 84, 72, 86, 70, 100, 79, 92, 30]

Ֆիզիկա = [40, 52, 65, 68, 45, 62, 46, 68, 75, 80, 99, 65, 17, 11]

18. Հավաքի՛ր տվյալներ ձեր բնակավայրի ամսական միջին տեղումների մասին և կազմի՛ր գծային դիագրամ՝ ներկայացնելով տեղումների քանակի փոփոխությունը:
19. Հավաքի՛ր տվյալներ համաշխարհային էներգիայի աղբյուրների տեսակների (օրինակ՝ արևային, քամի, ջրային, ածուխ, գազ) մասին և կազմի՛ր շրջանային դիագրամ՝ ներկայացնելով էներգիայի տեսակների հարաբերակցությունը՝ ըստ օգտագործման ծավալների:
20. Բաց աղբյուրներից հավաքի՛ր աշխարհի խոշորագույն սոցիալական մեդիա հարթակների (օրինակ՝ *Facebook, Instagram, TikTok, Youtube*) օգտատերերի քանակի մասին տվյալներ և կազմի՛ր շրջանային դիագրամ՝ ցույց տալով օգտատերերի տոկոսային բաշխումը:
21. Հավաքի՛ր տվյալներ տարբեր երկրների միջին աշխատավարձերի մասին և կազմի՛ր հիստոգրամ՝ ներկայացնելով աշխատավարձերի բաշխվածությունը:
22. Հետազոտի՛ր 7 խոշորագույն տեխնոլոգիական ընկերությունների (օրինակ՝ *Apple, Microsoft, Google, Amazon, Facebook*) վերջին տարվա տվյալները՝ հավաքելով եկամուտները (*Revenue*), աշխատողների քանակը (*Employees*) և շուկայական արժեքը (*Market Cap*): Կազմի՛ր պղպջակային դիագրամ, որտեղ՝
- X առանցքը կներկայացնի եկամուտը,
 - Y առանցքը կներկայացնի աշխատողների քանակը,
 - պղպջակի չափը կներկայացնի շուկայական արժեքը:
23. Գծի՛ր բևեռային գրաֆիկ (*polar plot*), որտեղ $r=1$ բոլոր անկյունների համար:
24. Գծի՛ր բևեռային գրաֆիկ, որտեղ $r=\theta$ շառավիղը հավասար է անկյանը բոլոր անկյունների համար:
25. Գծի՛ր բևեռային գրաֆիկ, որտեղ $r=\sin(\theta)$:
26. Գծի՛ր բևեռային գրաֆիկ, որտեղ $r=2+\cos(5\theta)$:
27. Ունենք տվյալներ Արագած լեռան գագաթին ուժեղ քամիների վերաբերյալ (20-30կմ/ժ)՝

Տևողություն (տարեկան)	Ուղղություն (աստիճան)	Կողմնացույցային ուղղությունը
------------------------	-----------------------	------------------------------

0.3 ժամ	247.5	SSW
1 ժամ	225	SW
1.1 ժամ	202.5	WSW
1.1 ժամ	180	W
0.5 ժամ	157.5	WNW



Կառուցի՛ր այս տվյալների համար «Քամու վարդը»:

28. Հետազոտի՛ր Երևանի (կամ այլ տարածաշրջանի) օդի մաքրության ինդեքսը և այդ տվյալներով կառուցի՛ր տուփային դիագրամ (*box plot*): Տվյալները կարող եք վերցնել օրինակ՝ *waqi.info* կայքից:
29. Հավաքի՛ր տվյալներ մեկ շաբաթվա ընթացքում որևէ բնակավայրի օրվա առավելագույն և նվազագույն ջերմաստիճանների մասին: Ներկայացրո՛ւ այն տարածքային գրաֆիկի միջոցով:

PANDAS ԳՐԱԴԱՐԱՆ

Pandas–ը Python ծրագրավորման լեզվի համար նախատեսված շատ հզոր գրադարան է, որը հիմնականում օգտագործվում է տվյալների վերլուծության և մշակման համար: Այն հարմար է կառուցված՝ շերտավորված տվյալների (օրինակ՝ աղյուսակների) հետ աշխատելու համար: Pandas գրադարանը ստեղծվել է Վես ՄըքՔենեյի կողմից 2008 թ.: ՄըքՔենեյն աշխատում էր AQR Capital Management-ում, որտեղ մեծ քանակի տվյալների վերլուծություն էր պահանջվում: Նա նկատեց, որ Python–ի այդ ժամանակվա գործիքները (ինչպես օրինակ՝ lists կամ dictionaries) չէին բավարարում տվյալների արագ և հարմար մշակման համար: ՄըքՔենեյը որոշում է առաջարկել Նոր գրադարան, որի հիմնական նպատակն էր ստեղծել հարմար, արդյունավետ և պարզ գործիք՝ աղյուսակային տվյալների վերլուծության համար: Նա մշակեց pandas գրադարանը: AQR Capital Management-ից հեռանալուց առաջ նա կարողանում է համոզել ղեկավարությանը գրադարանը դարձնել բաց կոդով:

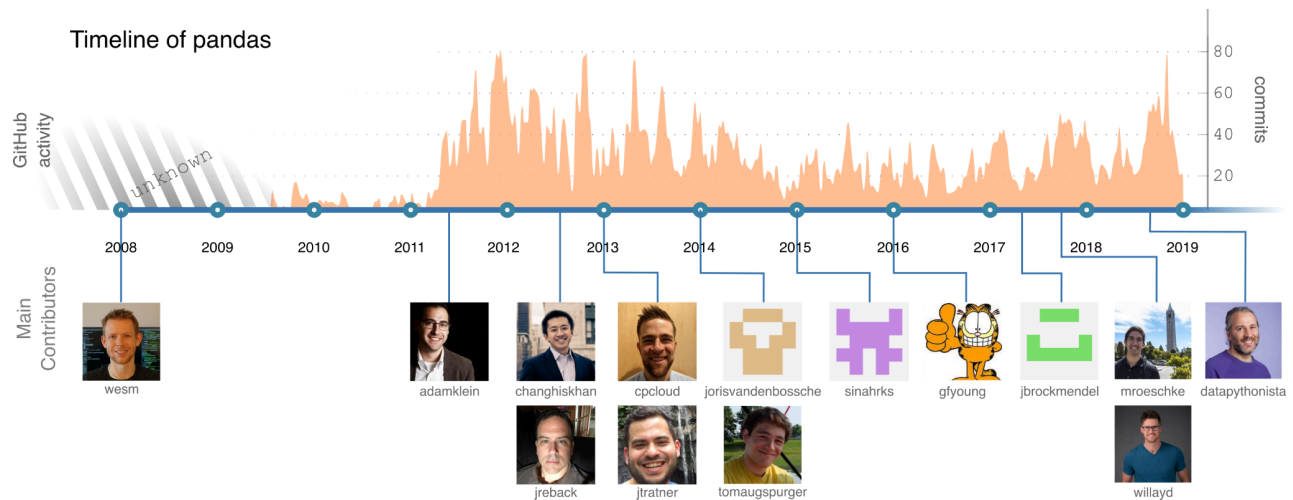


Վես ՄըքՔենեյ

Գրադարանի անվանումն առաջացել է *Panel Data* տերմիններից: ՄըքՔենեյը կրճատել է տերմինները՝ գրադարանի անվանումը դարձնելով *pandas*, որպեսզի այն ավելի հիշվող լինի:

Pandas-ի առավելությունները

- Հարմար և ինտուիտիվ՝ հեշտ է հասկանալ և օգտագործել, նույնիսկ սկսնակների համար:
- Արդյունավետություն՝ ավելի արագ է աշխատում մեծ տվյալների վրա, քան Python-ի ստանդարտ կառուցվածքները:
- Բազմաֆունկցիոնալություն՝ թույլ է տալիս հեշտությամբ ընտրել, ֆիլտրել, խմբավորել, մաքրել տվյալները:
- Տվյալների տարբեր աղբյուրներից ներմուծում՝ կարող է ներմուծել CSV, Excel, SQL, JSON և այլն տեսակի նիշքեր:
- Վիզուալիզացիայի ինտեգրացիա՝ հեշտ է օգտագործել տվյալների ներկայացման համար



(Matplotlib, Seaborn):

<https://se.ewi.tudelft.nl/desosa2019/chapters/pandas/>

Pandas-ի կիրառությունը

- Google՝ տվյալների վերլուծություն, AI/ML մոդելների նախապատրաստում:
- Facebook / Meta՝ տվյալագիտության և մեքենայական ուսուցման նախագծեր:
- Microsoft՝ Azure Data Science և բիզնես վերլուծական նախագծեր:
- Amazon՝ AWS և տվյալների վերլուծության ծառայություններ:
- Netflix՝ օգտատերերի նախապատվությունների վերլուծություն, recommendation systems:
- JPMorgan, Goldman Sachs, Bloomberg՝ ֆինանսական վերլուծություն, ռիսկերի մոդելավորում:
- Uber, Lyft՝ երթուղիների և ուղևորների տվյալների վերլուծություն:
- NASA, ESA՝ տիեզերական տվյալների վերլուծություն:
- CERN՝ մասնիկների ֆիզիկայի տվյալների մշակում:

- Harvard, MIT, Stanford` գիտական հետազոտություններ և պրոֆեսորական աշխատանքներ:

Տվյալների ներմուծում / արտահանում

- `pd.read_csv("file.csv")` → կարդում է csv ֆայլը:
- `df.to_csv("new.csv", index=False)` → պահում է csv ֆայլ:

Տվյալների ընդհանուր տեսք

- `df.head()` → առաջին 5 տողը:
- `df.tail()` → վերջին 5 տողը:
- `df.info()` → սյուների տեսակները, ոչ դատարկները:
- `df.describe()` → միջին, նվազագույն, առավելագույն, քառորդներ:

Սյուներ և տողեր

- `df["Column"]` → վերցնում է սյունը:
- `df[["Col1", "Col2"]]` → մի քանի սյուն:
- `df.loc[0]` → առաջին տողը (label հիման վրա):
- `df.iloc[0]` → առաջին տողը (index հիման վրա):

Ֆիլտրում

- `df[df["Age"] > 18]` → ֆիլտրում է բոլոր տողերը, որտեղ `Age > 18`:
- `df[(df["Age"] > 18) & (df["Gender"] == "F")]` → ֆիլտրում է մի քանի պայմանով:

Դասավորում և ինդեքսավորում

- `df.sort_values("Age")` → դասավորում է` ըստ տարիքի:
- `df.sort_values(["Age", "Salary"], ascending=[True, False])` → բազմակի է դասավորում:
- `df.set_index("ID")` → դնել ID-ն` որպես ինդեքս:

Բացակայող արժեքներ

- `df.isnull()` → ստուգում է դատարկները:
- `df.dropna()` → ջնջում է «դատարկ» պարունակող տողերը:
- `df.fillna(0)` → դատարկները փոխարինում է 0-ով:

Հաշվարկներ

- `df["Salary"].mean()` → միջին աշխատավարձ:
- `df["Age"].max()` → առավելագույն տարիք:
- `df["Score"].sum()` → գումար:

Խմբավորում

- `df.groupby("Gender")["Score"].mean()` → միջին միավոր` ըստ սեռի:
- `df.groupby(["Gender", "Class"]).size()` → հաշվարկ` ըստ երկու սյուների:

Միավորումներ

- `pd.concat([df1, df2])` → միավորում վերն-վերն:
- `pd.merge(df1, df2, on="ID")` → միավորում` ըստ ID սյունի:

Պատկերացում

- `df["Age"].plot(kind="hist")` → հիստոգրամ:
- `df.plot(kind="scatter", x="Age", y="Salary")` → սփռված գրաֆիկ:

Կիրառենք Pandas գրադարանի մեթոդներն օրինակների վրա:

Օրինակ 1,

Ունենք

	A	B	C	D	E	F	G
1	N	User	Age	email	city	signup_date	Gender
2	1	Mika Avanesyan	25	mik123@exa	Yerevan	2025-01-15	Male
3	2	Nare Hakobyan	30	anna456@ex	Gyumri	2024-12-10	Female
4	3	Gor Sargsyan	22	gor789@exa	Vanadzor	2025-03-05	Male
5	4	Lilit Grigoryan	28	lilit321@exa	Yerevan	2025-02-20	Female
6	5	Armen Manukyan	35	armen555@e	Goris	2024-11-30	Male
7	6	Narek Torosyan	27	narek777@ex	Yerevan	2025-04-01	Male
8	7	Mariam Khachatryan	24	mariam888@	Vanadzor	2025-03-22	Female
9	8	Hayk Babayan	31	hayk999@ex	Gyumri	2025-01-28	Male
10	9	Susan Avetisyan	29	susan222@e	Yerevan	2025-02-10	Female

աղյուսակը, որը պահպանված է `data.csv` անվանումով: Տպենք մի քանի տեղեկություններ աղյուսակի մասին՝

Python

```
import pandas as pd
```

```
# CSV ֆայլի ներմուծում
```

```
example = pd.read_csv("data.csv")
```

```
# Տպել առաջին 5 տողը
```

```
print("\n ♦ Առաջին 5 տողերը:")
```

```
print(example.head())
```

```
# Տպել ընդհանուր ինֆորմացիա
```

```
print("\n ♦ DataFrame-ի ինֆորմացիա:")
```

```
print(example.info())
```

```
# Տպել վիճակագրական ամփոփում
```

```
print("\n ♦ Վիճակագրական ամփոփում:")
```

```
print(example.describe())
```



```
# Տպել տողերի և սյուների քանակը
print("\n ♦ Չափերը (տողեր, սյուներ):")
print(example.shape)

# Տպել սյուների անունները
print("\n ♦ Սյուների անունները:")
print(example.columns)
```

Ծրագիրը կվերադարձնի՝

None

♦ Առաջին 5 տողերը:

	N	User	Age	email	city	signup_date	Gender
0	1	Mika Avanesyan	25	mik123@example.com	Yerevan	2025-01-15	Male
1	2	Nare Hakobyan	30	anna456@example.com	Gyumri	2024-12-10	Female
2	3	Gor Sargsyan	22	gor789@example.com	Vanadzor	2025-03-05	Male
3	4	Lilit Grigoryan	28	lilit321@example.com	Yerevan	2025-02-20	Female
4	5	Armen Manukyan	35	armen555@example.com	Goris	2024-11-30	Male

♦ DataFrame-ի ինֆորմացիա:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9 entries, 0 to 8

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

0	N	9 non-null	int64
1	User	9 non-null	object
2	Age	9 non-null	int64
3	email	9 non-null	object
4	city	9 non-null	object
5	signup_date	9 non-null	object
6	Gender	9 non-null	object

dtypes: int64(2), object(5)

memory usage: 632.0+ bytes

None

♦ Վիճակագրական ամփոփում:

	N	Age
count	9.000000	9.000000
mean	5.000000	27.888889
std	2.738613	3.951090
min	1.000000	22.000000
25%	3.000000	25.000000
50%	5.000000	28.000000
75%	7.000000	30.000000
max	9.000000	35.000000

- ♦ Չափերը (տողեր, սյուներ):
(9, 7)
- ♦ Սյուների անունները:
Index(['N', 'User', 'Age', 'email', 'city', 'signup_date', 'Gender'], dtype='object')
>>>

Օրինակ 2,
Տպենք email սյունը:

```
Python
import pandas as pd

# CSV ֆայլի ներմուծում
example = pd.read_csv("data.csv")

#Տպում ենք email սյունը
print("\n Էլեկտրոնային փոստի հասցեները")
print(example["email"])
```

Օրինակ 3,
Տպենք տարիքների միջինը:

```
Python
import pandas as pd

# CSV ֆայլի ներմուծում
example = pd.read_csv("data.csv")

#Տպում ենք user-ների տարիքների միջինը
print("\n Ծննդյան տարիքների միջինը")
print(example["Age"].mean())
```

Օրինակ 4,
Տպենք տարիքների միջինը՝ ըստ սեռի:

```
Python
import pandas as pd
```

```
# CSV ֆայլի ներմուծում
example = pd.read_csv("data.csv")

#Տպում ենք user-ների տարիքների միջինը, ըստ սեռի
print("ԴՆ Տարիքների միջինը")
print(example.groupby("Gender")["Age"].mean())
```

ԱՌԱՋԱԴՐԱՆՔՆԵՐ

Ստեղծի՛ր հետևյալ աղյուսակի CSV ֆայլը և լուծի՛ր խնդիրներն այս աղյուսակի համար.

	A	B	C	D	E	F	G	H
1	ID	Name	Age	Gender	Class	Math_Score	Python_Score	School
2	1	Anna	16	F	10	88	77	School1
3	2	David	17	M	11	92	72	School2
4	3	Mary	15	F	10	76	65	School2
5	4	John	17	M	12			School3
6	5	Lily	16	F	11	90	79	School3
7	6	Mark	17	M	12	70	90	School3
8	7	Susan	16	F	10		58	School1
9	8	Tom	18	M	12	68	80	School2
10	9	Emma	17	F	11	80		School2
11	10	Alex	17	M	12	89	77	School1

1. Ներմուծի՛ր տվյալները CSV ֆայլից և տպի՛ր առաջին 5 տողը:
2. Ներմուծի՛ր տվյալները CSV ֆայլից և տպի՛ր վերջին 2 տողը:
3. Տպի՛ր տողերի և սյուների քանակը:
4. Տպի՛ր բոլոր սյուների անունները:
5. Տպի՛ր Age սյան միջին արժեքը:
6. Տպի՛ր Age սյան միջին արժեքը՝ ըստ սեռի:
7. Տպի՛ր Age սյան միջին արժեքը՝ ըստ դպրոցի:
8. Տպի՛ր Math_Score սյան միջին ստանդարտ շեղումը:
9. Տպի՛ր Python_Score սյան միջին ստանդարտ շեղումը՝ ըստ դպրոցների:
10. Տպի՛ր Math_Score սյան նվազագույն արժեքը:
11. Տպի՛ր Python_Score սյան առավելագույն արժեքը:
12. Տպի՛ր Python_Score սյան ընդհանուր գումարը:
13. Հաշվի՛ր և տպի՛ր բոլոր թերի (NaN) արժեքների քանակը:
14. Լրացրո՛ւ դատարկ բջիջները միջին արժեքով:
15. Ավելացրո՛ւ նոր սյուն Average_Score, որը կլինի մաթեմատիկայի և Python առարկայի գնահատականների միջինը:
16. Տպի՛ր ամենամեծ միջին գնահատական ունեցողի աշակերտի տվյալները:

17. Ցուցադրի՛ր բոլոր այն աշակերտների տվյալները, ովքեր մաթեմատիկայից ստացել են 80-ից բարձր:
18. Ցուցադրի՛ր բոլոր այն աշակերտների տվյալները, ովքեր մաթեմատիկայից ստացել են 80-ից ցածր:
19. Ցուցադրի՛ր միայն 10-րդ դասարանցիների տվյալները:
20. Հաշվի՛ր և տպի՛ր յուրաքանչյուր դպրոցի մաթեմատիկայի միջին գնահատականը:
21. Դասակարգի՛ր տվյալները՝ ըստ մաթեմատիկայի գնահատականների աճման:
22. Տպի՛ր մաթեմատիկայի ամենաբարձր գնահատական ստացած 10-րդ դասարանցու տվյալները:
23. Տպի՛ր Python-ից 3 ամենաբարձր գնահատական ունեցողների տվյալները:
24. Հաշվի՛ր յուրաքանչյուր դպրոցի աշակերտների թիվը:

Կազմակերպությունը տվյալներ է հավաքել իր երկիր այցելած զբոսաշրջիկների մասին:
Այն ներկայացված է հետևյալ աղյուսակով՝

A	B	C	D	E	F	G	H	I
Index	Id	First Name	Last Name	Age	Gender	Country	Days	Total Purchase
1	ID_0001	Jacob	Mckenzie	24	Male	Tajikistan	6	2253
2	ID_0002	Tammy	Thomas	44	Female	Gana	3	3796
3	ID_0003	Jesse	Thompson	19	Female	Italy	14	4084
4	ID_0004	Martin	Guerrero	64	Male	Austria	11	1786
5	ID_0005	Samantha	Gonzalez	70	Female	Namibia	14	20774
6	ID_0006	Edward	Holden	30	Male	Tonga	13	15728
7	ID_0007	Sabrina	Morris	61	Female	Slovakia	5	9743
8	ID_0008	Joseph	Collins	64	Male	Namibia	11	2651
9	ID_0009	Joana	Velasquez	29	Female	Solomon Islands	16	13378
10	ID_0010	Megan	Rivera	31	Female	Indonesia	12	6512
11	ID_0011	Jorge	Rose	30	Male	Namibia	6	255
12	ID_0012	Diana	Wyatt	35	Female	Bhutan		9116
13	ID_0013	David	Nersisyan	28	Male	Armenia	5	10833
14	ID_0014	Raya	Mckinney	37	Female	Tanzania	18	3843
15	ID_0015	Karen	Walker	37	Male	Benin	18	
16	ID_0016	Stephan	Branch	44	Male	Indonesia	6	4872
17	ID_0017	Mark	Costa	23	Male	Honduras	1	171
18	ID_0018	Carmen	Lalayan	33	Female	Armenia	5	2156
19	ID_0019	Anthony	Moore	70	Male	Namibia	4	524
20	ID_0020	Andrea	Newman	65	Female	Gana	9	2834
21	ID_0021	Kakha	Anishvili	56	Male	Georgia	11	8543
22	ID_0022	Jane	Allen	22	Female	Gana	5	1686

Աղյուսակը ամբողջությամբ կարող եք ներբեռնել հետևյալ [հղմամբ](#)՝ csv ֆորմատով:

Կատարի՛ր 25-35 առաջադրանքները նշված csv ֆայլի համար:

25. Լրացրո՛ւ աղյուսակի դատարկ արժեքները տվյալ սյան միջին արժեքով:
26. Հաշվի՛ր և տպի՛ր, թե միջինում քանի օր են անցկացնում զբոսաշրջիկները տվյալ երկրում:
27. Հաշվի՛ր և տպի՛ր միջինում անցկացրած օրերի քանակը՝ ըստ սեռերի:
28. Հաշվի՛ր և տպի՛ր այն երկրի/երկրների անունները, որի քաղաքացիները ամենաշատն են գումար ծախսել:

29. Հաշվի՛ր և տպի՛ր այն երկրի/երկրների անունները, որի քաղաքացիները միջինում ամենաշատն են ծախսել:
30. Հաշվի՛ր և տպի՛ր այն երկրի/երկրների անունները, որտեղից ամենաշատ զբոսաշրջիկներն են եկել:
31. Գծի՛ր զբոսաշրջիկների անցկացրած օրերի նորմալ բաշխման գրաֆիկը:
32. Ներկայացրո՛ւ օրերի և ծախսած գումարի դիագրամը Scatter plot-ով:
33. Ներկայացրո՛ւ ծախսված գումարների տվյալները հիստոգրամով:
34. Ներկայացրո՛ւ երկրները և դրա քաղաքացիների ծախսած ընդհանուր գումարները Bar chart-ով:
35. Կառուցի՛ր DataAnalysis դասը: Այն օգտագործելու է վերը նշված csv ֆայլը:

Ատրիբուտներ

- file - ստանում է csv ֆայլի ուղին (path-ը):

Մեթոդներ

- `info()`՝ տպում է ամբողջ աղյուսակը:
- `clean_data()`՝ դատարկ արժեքները լրացնում է համապատասխան սյունի միջին արժեքով:
- `print(column_name)`՝ տպում է նշված սյունի եզակի արժեքները (առանց կրկնության):
- `add_new_data(new_dict)`՝ ստանում է հետևյալ տեսքի բառարան՝

Python

```
{ "First Name": name, "Last Name": last_name, "Age": age, "Gender": gender, "Country": country, "Days": days, "Total Purchase": purchase},
```

Ավելացնում է նոր տվյալներ csv ֆայլում: Ինդեքսն և ID-ն գեներացնում է ինքնուրույն:

Թեմա 3:

ԱՅԼ ԳՐԱԴԱՐԱՆՆԵՐ:
ՄԻՋԱԿԱՅՐԵՐ:

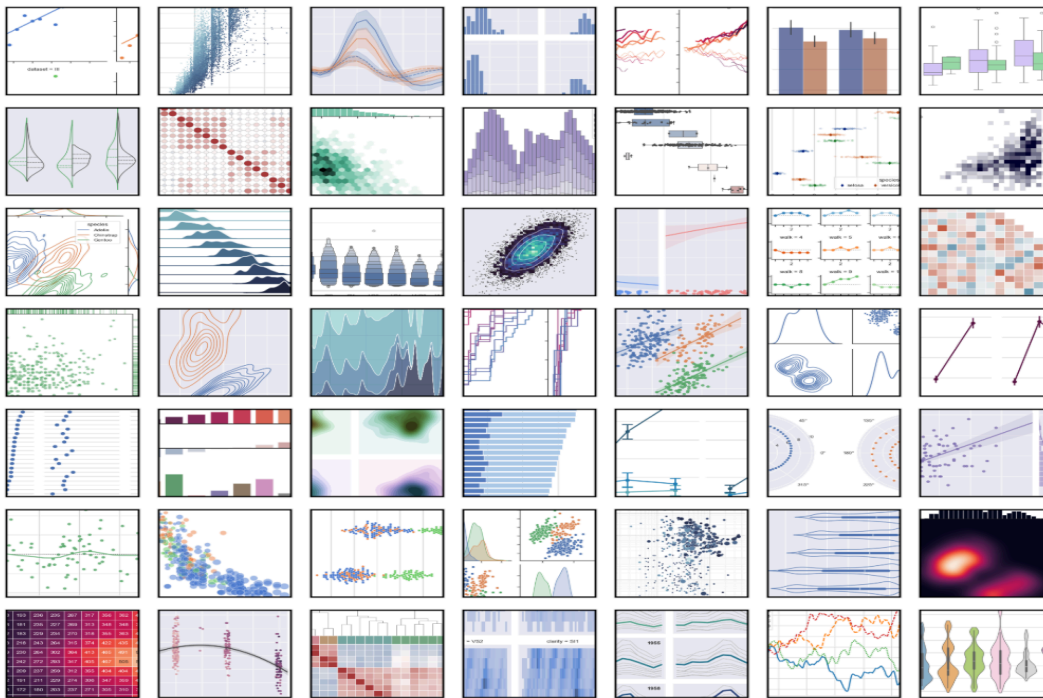
ԹԵՄԱ 3: ԱՅԼ ԳՐԱԴԱՐԱՆՆԵՐ: ՄԻՋԱԿԱՅՐԵՐ:

SEABORN ԳՐԱԴԱՐԱՆ

Seaborn-ը Python-ում տվյալների տեսապատկերման համար նախատեսված գրադարան է, որը

ստեղծվել է matplotlib-ի հիման վրա: Այն հիմնականում օգտագործվում է ստատիստիկ վիզուալիզացիայի համար՝ հեշտացնելով բարդ գրաֆիկների ստեղծումը մի քանի հրահանգով:

Seaborn գրադարանի տեսապատկերումների օրինակներ



#	Matplotlib	Seaborn
Ինչի համար է	Հիմնական տեսապատկերման գրադարան: Կարգավորումները հիմնականում արվում են մեխանիկորեն:	Հիմնված է matplotlib գրադարանի վրա: Հարմար է՝ արագ գրաֆիկներ ստանալու համար:
Կողի բարդություն	Ավելի շատ կոդ է պետք:	Ավելի քիչ կոդ է պետք:
Դիզայն	Պարզ դիզայն է, բոլոր փոփոխություններն արվում են մեխանիկորեն:	Ունի լռելյայն գեղեցիկ ու պատրաստի ոճեր:
Տվյալների աղբյուր	Լավ է աշխատում NumPy array-ներով:	Հարմար է Pandas DataFrame-ների համար:

Վիճակագրական գրաֆիկներ	Պետք է ինքդ հաշվարկես:	Կան պատրաստի ֆունկցիաներ՝ boxplot, pairplot, heatmap:
Ճկունություն	Շատ մեծ է, կարելի է ամեն ինչ փոխել:	Ավելի սահմանափակ է, բայց արագ:

Օրինակ,

Matplotlib գրադարան

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Պատահական սկյալներ
np.random.seed(42)
x = np.random.randn(100)
y = x * 2 + np.random.randn(100)

data = pd.DataFrame({"x": x, "y": y})

plt.figure(figsize=(7, 5))
plt.scatter(data["x"], data["y"], color="blue", alpha=0.6, edgecolors="black")
plt.title("Matplotlib Scatter Plot", fontsize=14)
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.show()
```

Seaborn գրադարան

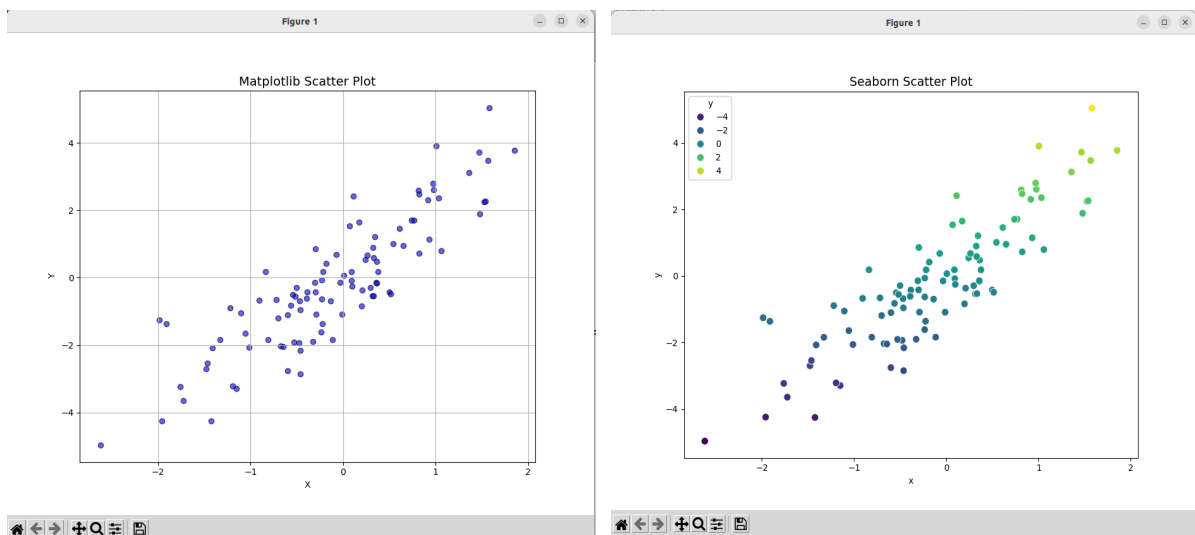
Python

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Պատահական սկյալներ
np.random.seed(42)
x = np.random.randn(100)
y = x * 2 + np.random.randn(100)

data = pd.DataFrame({"x": x, "y": y})

plt.figure(figsize=(7, 5))
sns.scatterplot(data=data, x="x", y="y", hue="y", palette="viridis", s=70)
plt.title("Seaborn Scatter Plot", fontsize=14)
plt.show()
```



Seaborn գրադարանն իր մեջ ունի ստանդարտ տվյալներ (data set), որոնցից կարելի է օգտվել սովորելու ընթացքում հետևյալ հրամանի միջոցով`

Python

```
import seaborn as sns
sns.load_dataset("name")
```

“Name”-ի փոխարեն գրվում է համապատասխան dataset-ի անունը:

Օրինակ,

Dataset	Նկարագրություն	Հիմնական սյուներ
tips	Ռեստորանային հաշիվներ և թեյավճարներ:	total_bill, tip, sex, smoker, day, time, size
iris	Ծաղիկների չափումներ (դասական ML dataset):	sepal_length, sepal_width, petal_length, petal_width, species
penguins	Պինգվինների չափերը՝ ըստ տեսակի և կղզու:	species, island, bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, sex
flights	Ուղևորների քանակ (1949–1960):	year, month, passengers
diamonds	Ադամանդների չափումներ և գին:	carat, cut, color, clarity, depth, table, price
fmri	Ուղեղի ակտիվության տվյալներ ժամանակի ընթացքում:	timepoint, event, region, signal
titanic	Տիտանիկ նավի ուղևորների տվյալներ:	survived, pclass, sex, age, sibsp, parch, fare, embarked

Ներմուծենք Տիտանիկի dataset-ը և տպենք առաջին 10 տողը:

```
Python
import seaborn as sns

# Տիտանիկի dataset-ի ներմուծում
titanic = sns.load_dataset("titanic")
```

```
# Տպում ենք առաջին 10 տողերը, որպես աղյուսակ
print(titanic.head(10))
```

Արդյունքը՝

Python

```
>>> %Run 'example.py'
```

	survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	0	3	male	22.0	...	NaN	Southampton	no	False
1	1	1	female	38.0	...	C	Cherbourg	yes	False
2	1	3	female	26.0	...	NaN	Southampton	yes	True
3	1	1	female	35.0	...	C	Southampton	yes	False
4	0	3	male	35.0	...	NaN	Southampton	no	True
5	0	3	male	NaN	...	NaN	Queenstown	no	True
6	0	1	male	54.0	...	E	Southampton	no	True
7	0	3	male	2.0	...	NaN	Southampton	no	False
8	1	3	female	27.0	...	NaN	Southampton	yes	False
9	1	2	female	14.0	...	NaN	Cherbourg	yes	False

```
[10 rows x 15 columns]
```

```
>>>
```

PLOTLY ԳՐԱԴԱՐԱՆ plotly

Plotly-ն Python-ում օգտագործվող հայտնի գրադարան է: Ինտերակտիվ է՝ գրաֆիկներ, տեսապատկերումներ կառուցելու համար: Այն առանձնանում է իր դինամիկությամբ. գրաֆիկները կարելի է մոտեցնել, հեռացնել, պահել պատկեր ձևաչափով կամ HTML Ֆայլի մեջ:

Հիմնական առանձնահատկությունները

- Գրաֆիկների տեսակներ՝ Line, Bar, Scatter, Histogram, Pie, Box plot, Bubble chart, 3D plot և շատ ուրիշներ:

- Ինտերակտիվություն՝ խոշորացում, պտտում (3D գրաֆիկների դեպքում), տվյալների ցուցադրում հովերների վրա:
- Վեբ ինտեգրում՝ կարելի է ստեղծել ինտերակտիվ վիզուալիզացիաներ, որոնք HTML ֆայլի մեջ բացվում են բրաուզերում:
- Plotly Express (px)՝ հեշտացված սինտաքսով մոդուլ, որը թույլ է տալիս մեկ տողով սկսել բարդ գրաֆիկներ:
- Dash՝ Plotly-ի վրա հիմնված հավելվածների շրջանակ (framework), որի միջոցով կարելի է վեբ-հավելվածներ ստեղծել տվյալների վերլուծության համար:

Plotly Express հրամանների աղյուսակ

Գրաֆիկի տեսակ	Հրաման (Plotly Express)	Հիմնական պարամետրեր
Line Chart (գծային)	<code>px.line(df, x, y)</code>	<code>x, y, color, title, labels</code>
Bar Chart (սյունակաձև)	<code>px.bar(df, x, y)</code>	<code>x, y, color, barmode</code>
Scatter Plot (ցրված)	<code>px.scatter(df, x, y)</code>	<code>x, y, color, size, hover_data</code>
Histogram (հիստոգրամ)	<code>px.histogram(df, x)</code>	<code>x, color, nbins</code>
Pie Chart (շրջանաձև)	<code>px.pie(df, names, values)</code>	<code>names, values, color</code>
Box Plot (տուփային)	<code>px.box(df, x, y)</code>	<code>x, y, color, points</code>
Violin Plot (ջութակաձև)	<code>px.violin(df, x, y)</code>	<code>x, y, color, box, points</code>
Area Chart (տարածքային)	<code>px.area(df, x, y)</code>	<code>x, y, color</code>
Bubble Chart (պղպջակային)	<code>px.scatter(df, x, y, size=..., color=...)</code>	<code>x, y, size, color</code>
3D Scatter	<code>px.scatter_3d(df, x, y, z)</code>	<code>x, y, z, color, size</code>
3D Surface	<code>px.surface(z=...)</code>	<code>z (մատրից), color</code>
Heatmap (ջերմային քարտեզ)	<code>px.density_heatmap(df, x, y)</code>	<code>x, y, color_continuous_scale</code>

Timeline (ժամանակագիծ)

```
px.timeline(df, x_start,  
x_end, y)
```

```
x_start, x_end, y
```

Օրինակներ,

1.

Python

```
import plotly.express as px
```

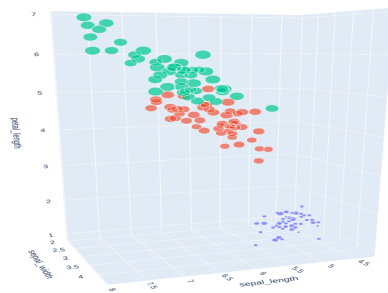
```
# Iris dataset
```

```
df = px.data.iris() # սա գրադարանում առկա օրինակ է
```

```
# 3D Scatter plot
```

```
fig = px.scatter_3d(df, x="sepal_length", y="sepal_width", z="petal_length",  
color="species", size="petal_width",  
title="Iris dataset - 3D Scatter")
```

```
fig.show()
```



2.

Python

```
import plotly.graph_objects as go
```

```
import numpy as np
```

```
# Կոնի պարամետրեր
```

```
theta = np.linspace(0, 2*np.pi, 50) # անկյուն
```

```
z = np.linspace(0, 2, 50) # բարձրություն
```

```
theta, Z = np.meshgrid(theta, z)
```

```
# Կոնի հավասարում (r նվազում է բարձրության հետ)
```

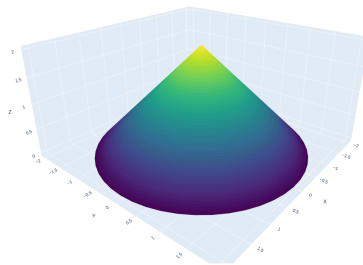
```

R = 2 - Z
X = R * np.cos(theta)
Y = R * np.sin(theta)

# Surface plot
fig = go.Figure(data=[go.Surface(x=X, y=Y, z=Z, colorscale="Viridis")])
fig.update_layout(title="Single Cone Example",
                  scene=dict(
                      xaxis_title="X",
                      yaxis_title="Y",
                      zaxis_title="Z"))

fig.show()

```



3. ժամանակային քարտեզ

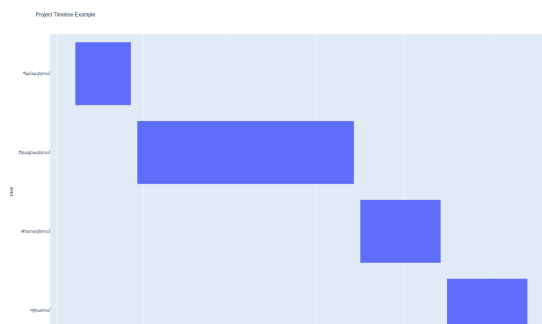
```

Python
import plotly.express as px
import pandas as pd

# Օրինակային project timeline
df = pd.DataFrame([
    dict(Task="Պլանավորում", Start="2025-01-01", Finish="2025-01-10"),
    dict(Task="Օրագրավորում", Start="2025-01-11", Finish="2025-02-15"),
    dict(Task="Թեսավորում", Start="2025-02-16", Finish="2025-03-01"),
    dict(Task="Կիրառում", Start="2025-03-02", Finish="2025-03-15")
])

# Timeline (Gantt chart)
fig = px.timeline(df, x_start="Start", x_end="Finish", y="Task",
                 title="Project Timeline Example")
fig.update_yaxes(autorange="reversed") # որպեսզի Tasks-ը վերևից ներքև լինին
fig.show()

```



ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Տրված է տղամարդկանց և կանանց բաշի ու հասակի տվյալները:

Python

```
"Height": [150, 160, 165, 170, 175, 180, 185, 190],  
"Weight": [50, 55, 60, 65, 72, 80, 85, 90],  
"Gender": ["F", "F", "F", "M", "M", "M", "M", "F"]
```

Գծի՛ր ցրված գրաֆիկ (scatter plot) Plotly Express գրադարանով:

2. Տրված է ամսական վաճառքի տվյալ: Գծի՛ր գծային գրաֆիկ (line chart), որտեղ x առանցքը ամիսներն են, y՝ վաճառքը:

Python

```
"Month": ["Jan", "Feb", "Mar", "Apr", "May"],  
"Sales": [120, 150, 170, 160, 180]
```

3. Տրված է վաճառքի տվյալ երկու խանութի համար: Գծի՛ր սյունակաձև գծապատկեր (bar chart), որտեղ x-ը՝ Month, y-ը՝ Sales, գույնը՝ ըստ Store:

Python

```
"Month": ["Jan", "Jan", "Feb", "Feb", "Mar", "Mar"],  
"Store": ["A", "B", "A", "B", "A", "B"],  
"Sales": [100, 120, 130, 140, 150, 160]
```

4. Տրված է ուսանողների գնահատականների տվյալները: Գծի՛ր հիստոգրամ՝ տրված տվյալներով:

Python

```
"Score": [55, 60, 72, 75, 78, 45, 92, 50, 55, 70, 80, 82, 85, 87, 90, 92, 95, 65, 70, ]
```


5. Տրված է երկու մրգերի՝ մանգոյի ("M") և կիվիի ("K"), վաճառքի տվյալները մի քանի օրվա ընթացքում:

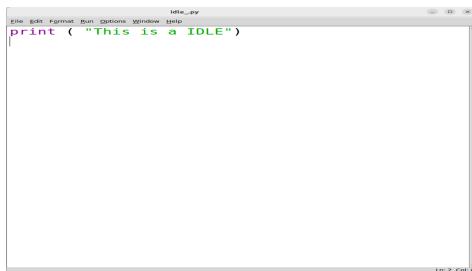
Python

```
"Group": [ "M", "K", "M", "K", "K", "M", "K", "M", "M", "M", "K", ],  
"Weight": [ 12, 15, 14, 18, 20, 19 , 24, 11, 14, 7 , 12 ]
```

Գծի՛ր տուփային գրաֆիկ և համեմատի՛ր վաճառքների տվյալները:

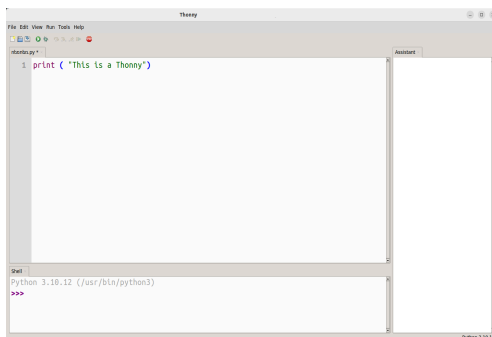
PYTHON ԼԵՃՎԻ ՄԻՋԱՎԱՅԻՐԵՐ IDLE

IDLE-ը (Integrated Development and Learning Environment) Python-ի ստանդարտ միջավայրն է, որն ինքնին կա Python-ի հետ միասին: Աշխատում է գրեթե բոլոր ՕՏ-երի հետ Windows, Linux, MacOS: Թեթև է ու պարզ օգտագործման համար: Չունի առանձնացված վրիպագերծման և կոդի ինքնաշարունակման համակարգեր:



THONNY

Thonny-ին ևս Python լեզվի ծրագրավորման անցանց (offline) միջավայր է: Ծրագիրն անվճար է, ունի վրիպագերծման (քայլ առ քայլ) մեթոդ և փոփոխականների արժեքների ցուցադրման պատուհան: Հարմար է սկսնակների համար՝ ծրագրավորում սովորելու նպատակով: Ծրագիրը կարելի է ներբեռնել thonny.org կայքից:



PYCHARM

Pycharm-ը պրոֆեսիոնալ կողավորման հարթակ է, աշխատում է առցանց և անցանց եղանակներով: Ունի վճարովի՝ **Professional Edition**, և անվճար՝ **Community Edition**, տարբերակները: Pycharm-ի հիմնական առանձնահատկություններ են՝

- Կոդի ավտոմատ լրացում (auto-completion), կոդի սխալների հայտնաբերում իրական ժամանակում, առաջարկներ բարելավման համար:
- Ունի կոդը վերակազմավորելու (rename, extract method, inline variable) հզոր գործիքներ:
- IDE-ի ներսում հնարավոր է օգտագործել terminal կամ interactive Python console:
- Ցանցային ֆունկցիաների, փոփոխականների, import-ների արագ որոնում և հասանելիություն:
- Կարելի է տեղադրել հավելումներ՝ Docker, Kubernetes, Markdown, Database tools և այլն:

Pycharm-ը հզոր գործիք է պրոֆեսիոնալ կողավորման համար, սակայն դժվար օգտագործելի է սկսնակների համար: Համեմատենք այս երեք միջավայրերը:

#	IDLE	Thonny	PyCharm
Տեղադրում	Կա Python-ի հետ:	Առանձին տեղադրում է պահանջում:	Առանձին տեղադրում է պահանջում:
Օգտագործման հարմարավետություն	Հարմար սկսնակների համար:	Հարմար սկսնակների համար:	Դժվար է սկսնակների համար, նախատեսված է պրոֆեսիոնալ կողավորման համար:
Արագություն	Շատ թեթև է:	Թեթև է:	Ծանր է:
Վրիպագերծում	Հատուկ գործիքներ չունի:	Քայլ առ քայլ գործողություններ, փոփոխականների արժեքի ցուցադրում:	Մասնագիտական վրիպագերծման գործիքներ:
Ինքնալրացում	Սահմանափակ է:	Միջին է:	Լավ համակարգ է:
Մեծ նախագծերի համար	Հարմար չէ:	Հարմար չէ:	Հարմար է:
Արժեք	Անվճար է:	Անվճար է:	Community՝ անվճար, Pro՝ վճարովի:



ANACONDA

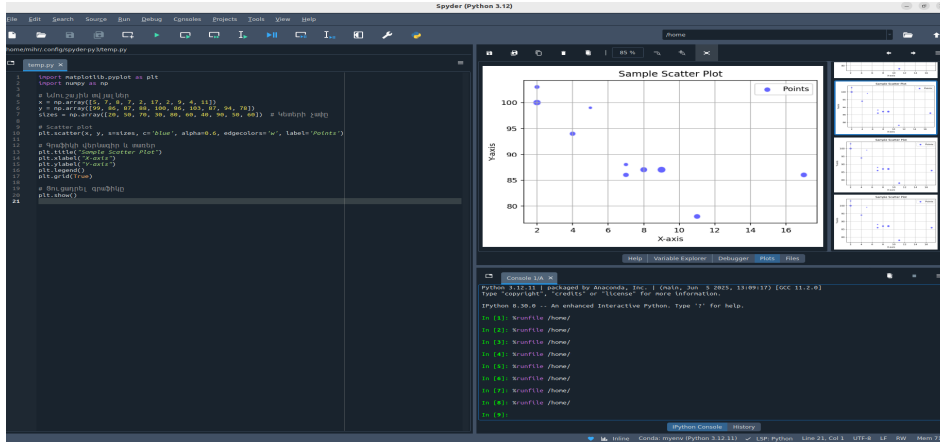
Anaconda-ն բաց կոդով (open-source) հարթակ է, որը նախատեսված է՝ տվյալագիտության (Data Science), մեքենայական ուսուցման (Machine Learning), արհեստական բանականության (AI) և մեծ տվյալների հետ աշխատելու համար: Ունի conda փաթեթների կառավարիչ, որով հեշտ է գրադարաններ տեղադրել, թարմացնել ու կառավարել: Ինքնուրույն ստեղծում է վիրտուալ միջավայրեր, որպեսզի տարբեր նախագծերն իրար չխանգարեն: Ներառում է ավելի քան 1500 գիտական գրադարան (numpy, pandas, scikit-learn, matplotlib և այլն), ինչպես նաև Jupyter Notebook, Spyder IDE, և այլ գործիքներ: Այս միջավայրը կարելի է ներբեռնել Anaconda-ի պաշտոնական կայք էջից <https://www.anaconda.com/>:

conda հրամանների փաթեթներ

Գործողություն	Հրաման
Նոր միջավայր ստեղծել:	<code>conda create -n myenv python=3.12</code>
Միջավայր ակտիվացնել:	<code>conda activate myenv</code>
Միջավայր ապասկտիվացնել:	<code>conda deactivate</code>
Բոլոր միջավայրերը ցուցադրել:	<code>conda env list</code>
Միջավայր ջնջել:	<code>conda remove -n myenv --all</code>
Փաթեթ տեղադրել:	<code>conda install numpy</code>
Փաթեթ թարմացնել:	<code>conda update numpy</code>
Բոլոր փաթեթները թարմացնել:	<code>conda update --all</code>
Փաթեթ ջնջել:	<code>conda remove matplotlib</code>
Տեղադրված փաթեթների ցուցակ:	<code>conda list</code>
Conda-ի տարբերակը տեսնել:	<code>conda --version</code>
Ընդհանուր տեղեկություն:	<code>conda info</code>
Միջավայրը export անել (yaml):	<code>conda env export > env.yaml</code>
Միջավայր ստեղծել yaml-ից:	<code>conda env create -f env.yaml</code>

Anaconda-ն հարթակ է, որտեղ կող գրել չեք կարող (չի հանդիսանում IDE): Դրա համար պետք է օգտագործել Spyder, Jupyter Notebook միջավայրերից մեկը:

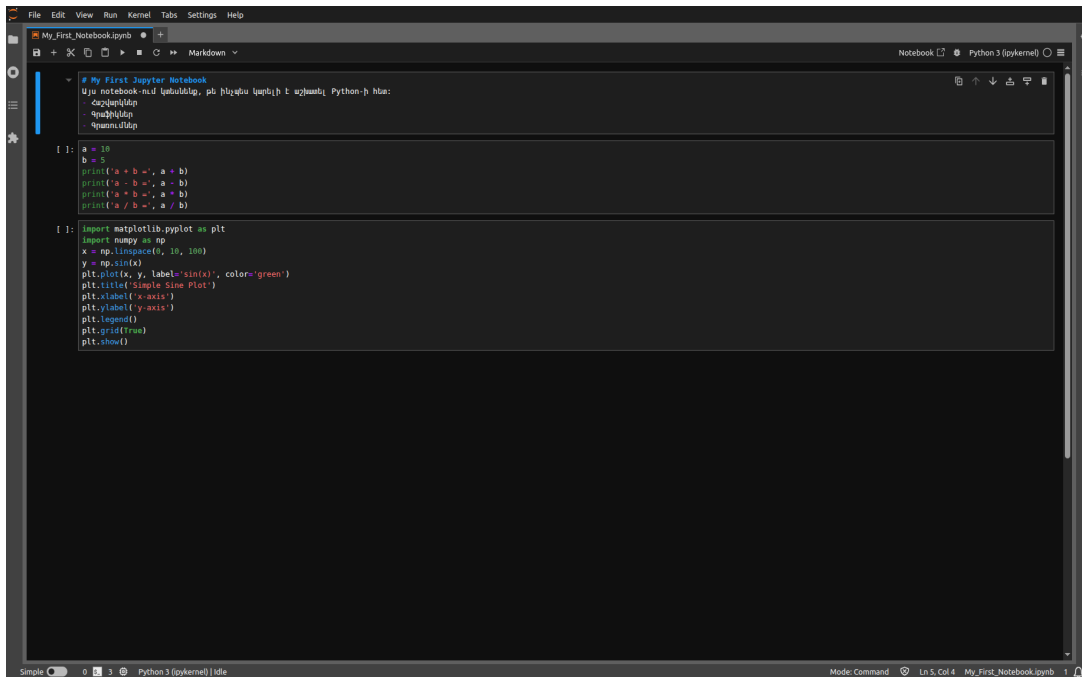
Spyder միջավայրում գրված պարզ կոդ՝



JUPYTER NOTEBOOK



Jupyter Notebook-ն ինտերակտիվ վեբ միջավայր է, որը թույլ է տալիս Python-ով (և այլ լեզուներով) գրել, գործարկել և փաստաթղթավորել կոդը մեկ վահանակում (notebook)՝ տեքստով, գրաֆիկներով, աղյուսակներով և մաթեմատիկական բանաձևերով: Այն շատ հարմար է տվյալների վերլուծության, գիտական հաշվարկների, մեքենայական ուսուցման և կրթության համար: Jupyter Notebook-ը տեղադրվում է anaconda միջավայրի հետ միասին, սակայն հնարավոր է առանձին տեղադրել: Ավելի մանրամասն կարելի է կարդալ <https://jupyter.org/> կայքում:



KAGGLE

Kaggle-ը Google-ին պատկանող հարթակ է, որը նախատեսված է մեքենայական ուսուցման, արհեստական բանականության և տվյալագիտության ուսումնասիրության և գործնական փորձարկումների և մրցույթների իրականացման համար: Այն հնարավորություն է տալիս ինչպես սկսնակներին, այնպես էլ փորձառու մասնագետներին աշխատել իրական տվյալների հետ և զարգացնել հմտությունները:

Kaggle-ը հիմնական հնարավորությունները

1. **Մրցույթներ**

Kaggle-ը հայտնի է իր մրցույթներով: Կազմակերպությունները տեղադրում են իրական խնդիրներ (օրինակ՝ կանխատեսել հիվանդություն, ճանաչել պատկերներ, կանխատեսել վաճառքներ): Մասնակիցները ստեղծում են մոդելներ և փորձում ստանալ ամենաբարձր ճշգրտությունը:

2. **Տվյալների հավաքածուներ**

Kaggle-ում կան միլիոնավոր տվյալների հավաքածուներ (CSV, JSON և այլն): Օգտատերերը կարող են ներբեռնել կամ կիսվել իրենց տվյալներով:

3. **Kernels / Notebooks (Նոթբուքեր)**

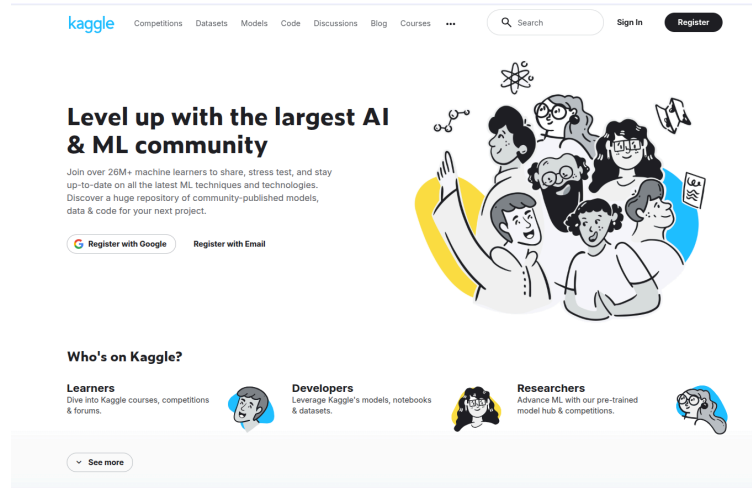
Kaggle-ն ունի ամպային միջավայր (cloud), որտեղ կարելի է Python կամ R կոդ գրել: Կարելի է նաև անվճար GPU օգտագործել՝ մոդելներ վարժեցնելու համար:

4. Դասընթացներ

Kaggle Learn բաժնում կան կարճ ու պրակտիկ դասեր, որտեղ կարճ ժամանակում կարելի է սովորել Python, Pandas, Machine Learning basics:

5. Համայնք

Kaggle-ում կան ֆորումներ, որտեղ մարդիկ կիսվում են փորձով և լուծումներով:



Kaggle -ի մասին կարող եք ավելի մանրամասն ուսումնասիրել պաշտոնական կայք էջում <https://www.kaggle.com>: Kaggle-ից կարելի է ներբեռնել տվյալներ CSV ֆորմատով և կատարել առաջադրանքներ դրանց հետ:

ԱՌԱՋԱԴՐԱՆՔՆԵՐ

Ներբեռնի՛ր տեսախաղերի մասին տվյալները հետևյալ հղումով՝

<https://www.kaggle.com/datasets/gregorut/videogamesales>

1. Գրի՛ր ծրագիր, որը կստանա այս տվյալների հավաքածուն CSV ֆորմատով և հիստոգրամի միջոցով կներկայացնի յուրաքանչյուր տասնամյակում թողարկված խաղերի քանակները:
2. Գրի՛ր ծրագիր, որը կստանա այս տվյալների հավաքածուն CSV ֆորմատով և շրջանային դիագրամով (pie chart) կներկայացնի խաղային օգտագործվող խաղային հարթակները (platform):
3. Գրի՛ր ծրագիր, որը կստանա այս տվյալների հավաքածուն CSV ֆորմատով և սյունյակաձև դիագրամով կներկայացնի տրված խաղային ժանրերը՝ ըստ քանակների:
4. Գրի՛ր ծրագիր, որը կստանա այս տվյալների հավաքածուն CSV ֆորմատով և ցրված գրաֆիկով կներկայացնի եվրոպական երկրներում վաճառքի (EU_sales) և ճապոնիայում վաճառքի (JP_sales) քանակները:
5. Գրի՛ր ծրագիր, որը կստանա այս տվյալների հավաքածուն CSV ֆորմատով, կհաշվի և կտպի այլ տարածաշրջանների (Other_Sales) վաճառքի՝
 - մեծագույն և փոքրագույն արժեքները,
 - միջինը, մեդիան,
 - վարիացիան և ստանդարտ շեղումը:

ՎԻՐՏՈՒԿԱԼ ՄԻՋԱՎԱՅՐ (PYTHON VENV)



(venv)

Python-ի venv-ն իրենից ներկայացնում է մեկուսացված միջավայր համակարգչում կամ ամպային տարածքում, որտեղ կարելի է գործարկել և փորձարկել տարատեսակ Python նախագծեր: Իրականում, երբ օգտագործում ենք Jupyter Notebook-ներ Google Colab-ում, Kaggle-ում կամ AWS Cloud9-ի նման ամպային տարածքներում, փաստացի աշխատում ենք մեկուսացված Python միջավայրում: «Virtual Environment» (վերիրական միջավայր) հասկացությունն այս դեպքում նշանակում է մեկուսացված միջավայր, որտեղ կարելի է իրականացնել որևէ համակարգի կամ ծրագրավորման լեզվի աշխատանք՝ առանց ազդելու համակարգի կամ այլ միջավայրերի և տարբերակների վրա: Վիրտուալ միջավայրում կարելի է ստեղծել առանձնացված ծրագրեր, գրադարաններ կամ պահաջնվող ծրագրային լրակազմ:

Python-ի venv-ի ստեղծումը

Համակարգչում Python-ի վիրտուալ միջավայր ստեղծելու համար հարկավոր է բացել հրահանգների տողը կամ վահանակը (command prompt, terminal) այն թղթապանակից, որտեղ ցանկանում եք ստեղծել ծրագիրը և մուտքագրել այս հրամանը՝

```
None
Windows
C:\Users\Your Name> python -m venv genaiproject
```

```
None
Linux/MacOS
$ python -m venv genaiproject
```

Այս հրամանը ստեղծում է myfirstproject անունով վիրտուալ միջավայրը, որն իրենից ներկայացնում է պանակ, և այն իր մեջ ներառում է հետևյալը՝

```
Shell
genaiproject
  Include
  Lib
  Scripts
  .gitignore
  pyvenv.cfg
```

Վիրտուալ միջավայրն օգտագործելու համար հարկավոր է ակտիվացնել այն:

```
Shell
Windows
C:\Users\Your Name> genaiproject\Scripts\activate
```

```
Shell
Linux/MacOS
$ source genaiproject/bin/activate
```

Ակտիվացումից հետո հրահանգների տողը (prompt) կփոխվի և ցույց կտա, որ այժմ աշխատում եք ակտիվ վիրտուալ միջավայրում:

```
genaipop-os:~$ python -m venv genaiproject
genaipop-os:~$ cd genaiproject/
genaipop-os:~/genaiproject$ ls
bin  include  lib  lib64  pyvenv.cfg
genaipop-os:~/genaiproject$ cd ..
genaipop-os:~$ source genaiproject/bin/activate
(genaiproject) genaipop-os:~$
```

Ակտիվացնելուց հետո արդեն կարող եք տեղադրել փաթեթները՝ օգտագործելով pip-ը: Տեղադրենք plotly գրադարանը՝

```
Shell
pip install plotly
```

```
(genaiproject) genaipop-os:~$ pip install plotly
Collecting plotly
  Downloading plotly-6.3.0-py3-none-any.whl (9.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.8/9.8 MB 3.8 MB/s eta 0:00:00
Collecting narwhals>=1.15.1
  Downloading narwhals-2.5.0-py3-none-any.whl (407 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 407.3/407.3 KB 1.5 MB/s eta 0:00:00
Collecting packaging
  Using cached packaging-25.0-py3-none-any.whl (66 kB)
Installing collected packages: packaging, narwhals, plotly
Successfully installed narwhals-2.5.0 packaging-25.0 plotly-6.3.0
(genaiproject) genaipop-os:~$
```


Ինչպես տեսնում ենք նկարից, Plotly-ն տեղադրված է: Գրենք մի պարզ ծրագրի օրինակ՝ ցույց տալու համար, թե ինչպես կարող ենք օգտագործել Plotly-ն վիրտուալ (venv) միջավայրում: Ստեղծի՛ր հետևյալ ծրագիրը և պահի՛ր որպես `plot_example.py`:

Python

```
import plotly.graph_objects as go

# Պարզ գծապատկերային սվյալներ
x_values = [1, 2, 3, 4, 5]
y_values = [10, 15, 13, 17, 14]

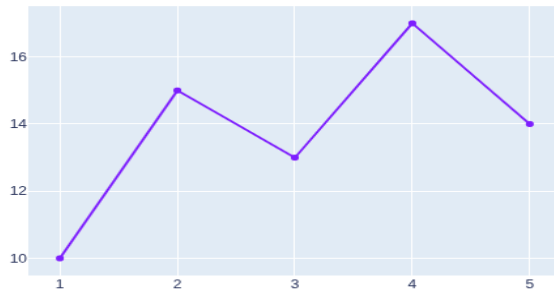
# Ստեղծում ենք գրաֆիկը
fig = go.Figure(data=go.Scatter(x=x_values, y=y_values, mode='lines+markers'))

# Ցուցադրում
fig.show()
```

Ծրագրի գործարկում

Shell

```
python plot_example.py
```



Ծրագիրը գործարկելուց համակարգչում կբացի վեբ-գլխարկիչի պատուհան, որտեղ կտեսնեք գծապատկերն ու նշումները (markers):



Docker-ը ծրագրային պլատֆորմ է, որը թույլ է տալիս փաթեթավորել, տարածել և աշխատեցնել ծրագրերը կոնտեյներների մեջ: Կոնտեյները թեթև, մեկուսացված միջավայր է, որը պարունակում է ծրագրի համար անհրաժեշտ ամեն ինչ՝ կոդ, գրադարաններ, dependencies (պահանջվող

ծրագրաշարեր), կարգավորումներ (config): Այսպիսով՝ Docker-ի միջոցով կարելի է ստեղծել միջավայր, որը նույն կերպ կաշխատի համակարգչում, սերվերի վրա և ամպային հարթակներում: Docker-ի մասին ավելի մանրամասն կարող եք ծանոթանալ այս հղումով docker-curriculum.com կամ Docker-ի պաշտոնական կայքում docker.com:

Համեմատենք վիրտուալ միջավայրերը, վիրտուալ մեքենաները և Docker-ը:

Բնութագիր	Docker	Վիրտուալ մեքենա (VM)	Python venv
Օպերացիոն համակարգ/Միջավայր	Կիսում է համակարգչի միջուկը:	Յուրաքանչյուր VM-ն ունի իր OS-ը:	Մեկուսացնում է Python միջավայրը:
Չափս (Size)	Թեթև է (մինչև մի քանի հարյուր MB):	Ծանր է (GB):	Ծառ փոքր է (մի քանի MB-ից պակաս):
Արագություն	Ծառ արագ է (վայրկյաններ):	Ավելի դանդաղ է:	Դանդաղ է:
Ռեսուրսների օգտագործում	Քիչ RAM և CPU:	Ավելի շատ RAM և CPU:	Նվազագույն (միայն Python-ի համար):
Մեկուսացում	Kernel մակարդակով:	Լիարժեք մեկուսացում (ավելի ապահով):	Python-ի գրադարանների և տարբերակների մեկուսացում:
Տեղափոխելիություն	Ծառ հեշտ է (Docker image ամեն տեղ աշխատում է):	Ավելի դժվար է՝ կախված է հիպերվայզոր-ից:	Հեշտ է (աշխատում է ցանկացած համակարգչում):
Օգտագործման դեպքեր	Միկրոսերվիսներ, DevOps, CI/CD, deploy:	Լիարժեք օպերացիոն համակարգեր, սերվերներ:	Պահանջվող ծրագրաշարերի կառավարում:
Պաշտպանվածությունը	Քիչ պաշտպանված է:	Ունի բարձր պաշտպանվածություն:	Միայն Python փաթեթների կառավարում:

Docker-ի մասին ավելին կարող եք ուսումնասիրել այստեղ՝ [geeksforgeeks.org](https://www.geeksforgeeks.org)

The screenshot shows the Docker Desktop application window. The left sidebar contains navigation options: Containers (selected), Images, Volumes, Builds, Dev Environments, Docker Scout, Docker Hub, and Extensions. The main area is titled 'Containers' and displays a table of running containers. At the top right, it shows 'Container CPU usage' as 0.01% / 1200% (12 CPUs allocated) and 'Container memory usage' as 249.6MB / 7.47 GB. A 'Show charts' button is also present. Below the table, a status bar indicates 'Engine running', 'RAM 2.20 GB', 'CPU 0.22%', 'Disk 3.62 GB used (limit 58.37 GB)', a 'Terminal' link, and the version 'v4.40.0'.

☐	Name	Container ID	Image	Port(s)	Last started	CPU %	Actions
☐	splashy-whale	43218j8jf3411	free-willy:latest	8000:8000	1 hour ago	0%	
☐	barnacle-bob	70398a3bdf55	bikini-bottom	-	2 hours ago	0.22%	
☐	yarr-matey	87324y65ff32	blue-beard	-	4 hours ago	0%	
☐	shrimp-clamstastic	34655h3ulf43	cocktail-sauce	-	10 hours ago	0%	
☐	oswald-west	57438h2gff89	cannon-beach	-	1 day ago	0%	
☐	endless-summer	5823aa342f5i	surfs-up	-	1 day ago	0%	

Թեմա 4:

ՍԽԱԼՆԵՐԻ

ՀԱՆԳՈՒՑԱԼՈՒԾՈՒՄ

ԹԵՄԱ 4: ՄԻԱԼՆԵՐԻ ՀԱՆԳՈՒՑԱԼՈՒԾՈՒՄ

ՄԻԱԼՆԵՐԻ ՏԵՍԱԿՆԵՐ

Ծրագրավորման լեզվի սխալներն այն իրավիճակներն են, երբ կոդն այլևս չի կարող շարունակել աշխատել և ընդհատվում է կամ չի կատարում ակնկալվող գործողությունը:

1. Շարահյուսության սխալներն (Syntax Errors) առաջանում են, երբ կոդի կառուցվածքը սխալ է:

```
Python
print("Hello World"
```

Սխալի մասին հաղորդագրություն՝ **SyntaxError: '(' was never closed**

```
Python
print("Hello World") a
```

Սխալի հաղորդագրություն՝

```
print("Hello World") a
                        ^
SyntaxError: invalid syntax
```

```
Python
a=10
if a > 5
    print("Ավելի մեծ է քան 5")
```

Սխալի հաղորդագրություն՝

```
if a > 5
    ^
SyntaxError: expected ':'
```

2. Գործարկման սխալներն (Runtime Errors) առաջանում են, երբ ծրագիրը ճիշտ է գրված, բայց գործարկման ժամանակ որոշակի սխալ է լինում:

- Զրոյով բաժանում (ZeroDivisionError):

- Չհայտարարած փոփոխականի օգտագործում (NameError):
- Ցուցակում գոյություն չունեցող ինդեքսի դիմում (IndexError):

Օրինակներ

```
Python
a=10
b = 0
if a>b:
    print (a/b)
```

Սխալի հաղորդագրություն`

```
print (a/b)
ZeroDivisionError: division by zero
```

```
Python
num=10
print ( number*5)
```

Սխալի հաղորդագրություն`

```
print ( number*5)
NameError: name 'number' is not defined
```

```
Python
st = "Hello world"
print ( st[11])
```

Սխալի հաղորդագրություն`

```
print ( st[11])
IndexError: string index out of range
```

TRY-EXCEPT

Try-except կառուցվածքը Python-ում բացառությունների (errors) մշակման մեխանիզմ է, որը թույլ է տալիս կառավարել ծրագրի ընթացքում առաջացող սխալները` կանխելով ծրագրի ընդհատումը: Այն

ապահովում է, որ սխալների դեպքում ծրագիրը շարունակի գործել կամ կատարի համապատասխան գործողություններ:

Կառուցվածք

```
Python
try:
    # Կոդ, որը կարող է հանգեցնել սխալի
except:
    # Միայնների կառավարման կոդ
```

Օրինակ

```
Python
try:
    number = int(input("Մուտքագրե՛ք թիվ: "))
    result = 10 / number
    print(f"Արդյունքը՝ {result}")
except ZeroDivisionError:
    print("Միայն՝ գրոյով բաժանումը հնարավոր չէ:")
except ValueError:
    print("Միայն՝ խնդրում եմ մուտքագրել թիվ:")
```

Try-except-ի հետ կա նաև finally մասնիկը, որը մշտապես կատարվում է՝ անկախ նրանից սխալ առաջացել է, թե ոչ:

Օրինակ,

```
Python
try:
    file = open("example.txt", "r")
    data = file.read()
    print(data)
except FileNotFoundError:
    print("Ֆայլը չի գտնվել:")
finally:
    file.close()
    print("Ֆայլը փակվել է:")
```

ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Գրի՛ր ծրագիր, որը ստանում է երկու թիվ և տպում է դրանց արտադրյալը: Կառավարի՛ր թվի փոխարեն տեքստի ներմուծման սխալը (ValueError):
2. Գրի՛ր ծրագիր, որը կբացի օգտատիրոջ կողմից ներմուծված անունով \$այլը: Կառավարի՛ր նշված անունով \$այլ գոյություն չունենալու սխալը:
3. Գրի՛ր ծրագիր, որն օգտատիրոջ կողմից ստանում է ցուցակ ու ինդեքս և տպում է ցուցակի այդ ինդեքսով տարրը: Կառավարի՛ր նշված ինդեքսի գոյություն չունենալու սխալը:
4. Գրի՛ր ֆունկցիա, որը ստանում է ցուցակ և հաշվում է ցուցակի բոլոր տարրերի արտադրյալը: Կառավարի՛ր ցուցակում թվային արժեքներից բացի այլ արժեքների գոյության սխալը:
5. Գրի՛ր ծրագիր, որը ստանում է a , b բնական թվեր և տպում է a^b -ը: Կառավարի՛ր շատ մեծ ամբողջ թիվ ստանալու սխալը ValueError-ով:
6. Գրի՛ր ծրագիր, որն օգտատիրոջից ստանում է n թիվ և տպում է 3.14^n թիվը: Կառավարի՛ր շատ մեծ թիվ ստանալու սխալը՝ օգտվելով OverflowError-ից:

ՍԵՓԱԿԱՆ ԲԱՑԱՌՈՒԹՅՈՒՆՆԵՐԻ ՍՏԵՂԾՈՒՄ

Ստեղծենք սեփական բացառություն (exception)՝ օգտագործելով ValueError-ը: Ենթադրենք, որ մեզ պետք է կառավարել բացասական թվի ներմուծման սխալը:

Python

```
def check_value(value):
    if value < 0:
        raise ValueError("Բացասական արժեքը թույլատրելի չէ")
    return f"Արժեքը ճիշտ է՝ {value}"

# Օգտագործում
try:
    result = check_value(-5)
    print(result)
except ValueError as e:
    print(f"ValueError: {e}")
```

7. Գրի՛ր ծրագիր, որը ստանում է օգտատիրոջ տարիքը և տպում է այն: Ստեղծի՛ր սեփական բացառում, որը կկառավարի 0 - 120 միջակայքից դուրս այլ թվի ներմուծման սխալը:
8. Գրի՛ր ծրագիր, որը ստանում է հեռախոսահամար և տպում է այն: Ստեղծի՛ր սեփական բացառում, որը կկառավարի ոչ վավեր հեռախոսահամարի ներմուծումը (երբ սկսվում է ոչ 0 թվանշանով, ունի սխալ երկարություն, թվային նիշից բացի ունի այլ նշաններ):
9. Գրի՛ր ծրագիր, որը ստանում է օգտատիրոջ անունն ազգանունը և տպում է այն: Ստեղծի՛ր սեփական բացառում, որը կկառավարի անվան ու ազգանվան ճիշտ մուտքագրումը:
10. Գրի՛ր ծրագիր, որը ստանում է չափահաս օգտատիրոջ տարիքը և տպում է այն: Ստեղծի՛ր սեփական InvalidAgeError, որ կկառավարի 18-120 միջակայքից դուրս այլ թվի ներմուծումը:

ԼՈՒԾՈՒՄ

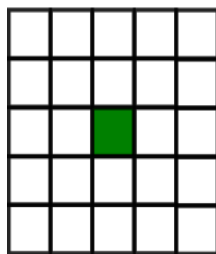
Python

```
# Սեփական բացառության ստեղծում
class InvalidAgeError(Exception):
    """Տարիքը սխալ է"""
    pass

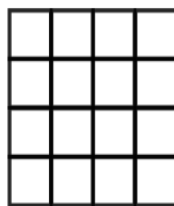
# Տարիքի ստուգման ֆունկցիա
def check_age(age):
    if age < 18 or age > 120: # Սահմանափակումներ տարիքի արժեքի համար
        raise InvalidAgeError("Տարիքը պետք է լինի 18-ից 120-ի միջև") # Բացառություն
        # Գեներում
    return f"Տարիքը վավեր է՝ {age}"

# Օգտագործում
try:
    result = check_age(190) # Փորձում ենք ստուգել սխալ տարիք
    print(result)
except InvalidAgeError as e: # Մշակում ենք InvalidAgeError բացառությունը
    print(f"Սխալ: {e}")
```

11. Գրի՛ր ծրագիր, որը ստանում է բանկային քարտից ելքագրվող գումարի չափը և տպում է այն էկրանին: Ստեղծի՛ր սեփական WithdrawalAmountError, որը կկառավարի 500.000-ից ավել և 1000-ին ոչ պատիկ թվերի մուտքագրման սխալը:
12. Գրի՛ր ծրագիր, որը ստանում է քառակուսաձև ցանցի չափը տողի տեսքով և տպում է դրա կենտրոնի կոորդինատը: Օրինակ՝ 5 x 5 ցանցի կենտրոնի կոորդինատը (3,3) է: Ստեղծի՛ր սեփական CenterExistenceError, որը կկառավարի ցանցի քառակուսաձև չլինելու և կենտրոն չունենալու սխալները:



Կենտրոն չունի



ՎՐԻՊԱՁԵՐՈՒՄԱՆ ԳՈՐԾԻՔՆԵՐ, ԵՒ ՏՐԱՄԱԲԱՆԱԿԱՆ ՍԽԱԼՆԵՐԻ ՎԵՐԼՈՒԾՈՒԹՅՈՒՆ

```
import pdb
```

```
pdb.set_trace()
```

հրամանը դադարեցնում է կոդը տվյալ տողում: Python 3.7-ից սկսած՝ կարելի է օգտագործել breakpoint() հրամանը՝ առանց որևէ մոդուլ ներմուծելու: Օրինակ՝

Python

```
x = 3
```

```
y = 7
```

```
breakpoint() # Կոդը դադարում է այստեղ
```

```
result = x * y
```

```
x = x / 2
```

```
div = x / y
```

```
print(div)
```

```
print ( result)
```

Հիմնական հրամանների ցանկ

Հրաման	Նկարագրություն
n	Անցնել հաջորդ տողին:
s	Մտնել վրիպագերծման ֆունկցիայի մեջ:
c	Շարունակել մինչև հաջորդ ընդհատման հրամանը:
q	Դուրս գալ վրիպագերծման ծրագրից:
p	Տպել փոփոխականի արժեքը: Օրինակ՝ p x: Կտպի x-ի արժեքը:
l	Տպել կոդի մոտակա տողերը:
b	Կանգառի կետ (breakpoint) ավելացնել: Օրինակ՝ b 15 կավելացնի 15-րդ տողում կանգառ:
cl	Ջնջել կանգառի կետերը (clear): Օրինակ՝ cl 1:
h	Ցուցադրել բոլոր հասանելի հրամանները:

Մի քանի հրամանների օգտագործում տերմինալում

```
Python
> /home/GenAI/example.py(5)<module>()
-> result = x * y
(Pdb) p x
3
(Pdb) n
> /home/GenAI/example.py(6)<module>()
-> x = x / 2
(Pdb) n
> /home/GenAI/example.py(7)<module>()
-> div = x / y
(Pdb) p x
1.5
(Pdb) l
   2   y = 7
   3
   4   breakpoint() # Կոդը դադարում է այստեղ
   5   result = x * y
   6   x = x / 2
   7   ->         div = x / y
   8
   9   print(div)
[EOF]
(Pdb) c
0.21428571428571427
```

13 - 18 խնդիրները պարունակում են ոչ ակնհայտ սխալներ, որոնց դեպքում ծրագիրը սխալ է աշխատում որոշակի արժեքների դեպքում: Թեստավորի՜ր անհրաժեշտ ծրագրերը՝ տարբեր արժեքներ տալով: Օգտվի՜ր *breakpoint()* հրամանից՝ միջանկյալ արժեքները հասկանալու համար:

13. Ունենք հետևյալ խնդիրը՝ ծրագիր, որը գտնում է ցուցակի մեծագույն տարրը՝

```
Python
def find_max_value(numbers):
    max_value = 0
    for num in numbers:
        if num > max_value:
            max_value = num
    return max_value
```

```
data = [-5, 10, -3, -8]
max_val = find_max_value(data)
print(f"The maximum value is: {max_val}")
```

Թեստավորի՞ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՞ր `breakpoint()` հրամանից:

14. Ունենք հետևյալ խնդիրը՝ ծրագիր, որը գտնում է ցուցակի փոքրագույն տարրը՝

```
Python
def find_minimum(numbers):
    min_val = numbers[0]
    for num in numbers:
        if num < min_val:
            min_val = num
        else:
            min_val = numbers[0]
    return min_val

data = [-3, 1, 4, 1, 5, 9]
result = find_minimum(data)
print(f"Minimum value: {result}")
```

Թեստավորի՞ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՞ր `breakpoint()` հրամանից:

15. Ունենք հետևյալ ծրագիրը, որը ստուգում է՝ արդյո՞ք նշված տարրը կա ցուցակում, թե ոչ՝

```
Python
def contains_value(numbers, value):
    found = False
    for num in numbers:
        if num == value:
            found = True
        else:
            found = False
    return "Yes" if found else "No"

data = [1, 15, 2, 3, 4, 5]
target = 5
result = contains_value(data, target)
```

```
print(f"Does the list contain {target}? {result}")
```

Թեստավորի՛ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՛ր `breakpoint()` հրամանից:

16. Ունենք ֆունկցիա, որը հեռացնում է ցուցակի կրկնվող տարրերը՝

Python

```
def remove_duplicates(numbers):
    result = []
    for num in numbers:
        if num not in result:
            result.append(num)
        else:
            result.remove(num)
    return result

data = [1, 24, 15, -30, 16]
result = remove_duplicates(data)
print(f"List without duplicates: {result}")
```

Թեստավորի՛ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՛ր `breakpoint()` հրամանից:

17. Ունենք ֆունկցիա, որը հաշվում է ցուցակի բոլոր զույգ թվերի գումարը՝

Python

```
def sum_even(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 0:
            total += num
        else:
            total = 0
    return total

data = [3, 5, -7, 20, 4, 6]
result = sum_even(data)
print(f"Sum of even numbers: {result}")
```

Թեստավորի՛ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՛ր `breakpoint()` հրամանից:

18. Ունենք ֆունկցիա, որը հաշվում է ցուցակի տարրերի հանրահաշվական միջինը՝

Python

```
def calculate_average(numbers):  
    total = 0  
    for num in numbers:  
        total += num  
    return total / len(numbers) if total > 0 else 0  
  
data = [-3, 20, -9, 7.4, 102, -65]  
result = calculate_average(data)  
print(f"Average: {result}")
```

Թեստավորի՞ր ծրագիրը՝ տալով տարբեր ցուցակներ: Օգտվի՞ր `breakpoint()` հրամանից:

ՍԻՍԱԼՆԵՐԻ ՀԱՂՈՐԴԱԳՐՈՒԹՅՈՒՆՆԵՐԻ ՍՏԵՂՃՈՒՄ USER-ՆԵՐԻ ՀԱՄԱՐ

Սիսալների հաղորդագրությունները պետք է օգտատիրոջն օգնեն ուղղել դրանք:

- Հաղորդագրությունը պետք է լինի պարզ և հասկանալի:
- Պետք է պարունակի տեղեկատվություն սիսալի պատճառի մասին:
- Պետք է առաջարկի լուծման տարբերակ կամ ուղղություն:

Օրինակ,

Python

```
def read_file(filename):  
    try:
```

```

with open(filename, "r") as file:
    return file.read()
except FileNotFoundError:
    return f"Միայն: '{filename}' ֆայլը չի գտնվել: Խնդրում եմf ստուգել ֆայլի
անունը:"
except PermissionError:
    return f"Միայն: '{filename}' ֆայլի վրա բավարար թույլտվություն չկա:"

```

19. Գրի՛ր ֆունկցիա, որ ստանում է երկու տող և վերադարձնում է դրանց երկարությունների հարաբերությունը: Ստեղծի՛ր հստակ սխալի հաղորդագրություն, թե ինչու չի կարելի դատարկ տող ներմուծել:
20. Գրի՛ր ծրագիր, որը ստանում է երկու թիվ և վերադարձնում դրանց արտադրյալը: Ստեղծի՛ր երկլեզու (հայերեն, անգլերեն) սխալի հաղորդագրություն, եթե օգտատերը ներմուծել է ոչ թվային արժեք:
21. Գրի՛ր ֆունկցիա, որը կստանա եռանկյան երեք կողմերը և կվերադարձնի դրա մակերեսը (օգտվի՛ր Ջերոնի բանաձևից): Ստեղծի՛ր սխալի հաղորդագրություն, եթե այդ երեք կողմերով եռանկյուն գոյություն չունի:

Թեմա 5:

ՏԱՐԲԵՐԱԿԻ ՎԵՐԱՀՍԿՄԱՆ ՀԱՄԱԿԱՐԳ

ԹԵՄԱ 5: ՏԱՐԲԵՐԱԿԻ ՎԵՐԱՅՍԿՄԱՆ ՀԱՄԱԿԱՐԳ

GIT-Ի ՄԱՍԻՆ

Git-ը տարբերակների վերահսկման համակարգ է (Version Control System – VCS), որը թույլ է տալիս ծրագրավորողներին հետևել կոդի փոփոխություններին, վերադառնալ հին տարբերակներին, աշխատել միաժամանակ մի քանի ճյուղերով և համագործակցել թիմով: Git-ի օգնությամբ յուրաքանչյուր փոփոխություն պահպանվում է որպես "commit", ինչը թույլ է տալիս ունենալ ամբողջ պատմությունը: Մինչ Git-ի հայտնվելը ծրագրավորողներն օգտագործում էին CVS, Subversion (SVN), Perforce և այլ համակարգեր: Դրանք հաճախ դանդաղ էին, բարդ և միշտ չէ, որ արդյունավետ էին մեծ նախագծերում:



2005 թ. **Linux-ի միջուկի (Linux Kernel)** ծրագրավորողների համայնքը կարիք ուներ հզոր, արագ և բաց աղբյուր ունեցող համակարգի: Այդ ժամանակ էլ առաջացավ Git-ի գաղափարը: Git-ի հեղինակը **Լինուս Տորվալդսն** է (Linus Torvalds)՝ Linux օպերացիոն համակարգի ստեղծողը: Տորվալդսը 2005 թ. ստեղծեց Git-ը՝ Linux-ի միջուկի մշակման ժամանակ ապահովելու համար արագություն, ճկունություն և վստահելի համակարգի առկայություն: Այդ ժամանակ Տորվալդսը չէր կարող պատկերացնել, որ հետագայում այն դառնալու է ամբողջ աշխարհի ծրագրավորողների ամենաշատ օգտագործվող գործիքը:

Git-ի հիմնական հնարավորությունները

- Արագություն՝ մեծ նախագծերում էլ Git-ը շատ արագ է:
- Բաշխվածություն՝ յուրաքանչյուր ծրագրավորող ունի ամբողջ պահոցի պատճենը:
- Անվտանգություն՝ տվյալները պահվում են հեշերով (SHA-1), ինչն ապահովում է փոփոխությունների ամբողջականությունը:
- Ճյուղավորում և միավորում (branching & merging)՝ հեշտ է ստեղծել ճյուղեր և միավորել դրանք:

Git-ի հետ աշխատելու քայլերը

1. Տեղադրում:
Ներբեռնիր ծրագիրը [Git-ի պաշտոնական կայքից](#) և տեղադրիր՝ քո համակարգչում:
Windows-ում հիմնականում օգտագործում են Git Bash կամ Command Prompt (cmd): Linux OS-ի դեպքում օգտագործում է համակարգի **Terminal-ը**:
2. Ստուգում:
Ստուգելու համար՝ արդյոք Git-ը ճիշտ է տեղադրված, թե ոչ համապատասխան տերմինալում գրիր՝
`git --version`
Եթե Git-ը ճիշտ է տեղադրված, կտեսնես սա՝ `git version 2.x.x`

3. Կարգաբերում:

Առաջին օգտագործումից առաջ պետք է սահմանել անունն ու էլ. փոստը՝

```
git config --global user.name "Քո Անունը"  
git config --global user.email "user@example.com"
```

4. Նոր նախագծի ստեղծում:

Նոր նախագիծը git-ով կառավարելու համար պետք է՝

- Գնալ այդ թղթապանակ (Folder):
- Բացել Git Bash և գրել `git init`:

Սա կստեղծի .git թաքնված թղթապանակ, որտեղ Git-ը կպահի բոլոր փոփոխությունները:

5. Փոփոխության պահպանում(commit)

Commit-ը նշանակում է ֆիքսել կատարված փոփոխությունները Git-ում: Երբ փոփոխություն ես անում ֆայլում, այն անմիջապես չի գրանցվում պատմության մեջ: Պետք է նախ git add, ապա git commit անել:

Օրինակ՝

```
git add example.py  
git commit -m "փոփոխել եմ Car կլասը"
```

Այս կողը նշանակում է՝

git add [example.py](#)՝ ավելացնում ենք փոփոխված ֆայլը:

git commit -m "փոփոխել եմ Car դասը"՝ պահպանում ենք փոփոխությունը՝ ավելացնելով բացատրող հաղորդագրություն:

6. Push հրաման

git push-ն օգտագործվում է՝ տեղական (local) փոփոխությունները GitHub (կամ այլ վիրտուալ պահոց) ուղարկելու համար:

Օրինակ՝

```
git push origin main
```

Այս կողը նշանակում է՝

origin՝ այն հեռավոր պահոցն է (GitHub-ը), որի հետ աշխատում ենք:

main՝ այն ճյուղն է, ուր ուղարկում ենք փոփոխությունները:

Եթե առաջին անգամ ենք push անում, կարող է պահանջվել այս հրամանը՝

```
git push --set-upstream origin main
```

Հիմնական հրամանների աղյուսակ

Հրաման	Նկարագրություն
<code>git init</code>	Ստեղծում է նոր պահոց:
<code>git clone <url></code>	Ներբեռնում է հեռավոր պահոցը:
<code>git status</code>	Ցույց է տալիս փոփոխված ֆայլերը:
<code>git add <ֆայլ></code>	Ավելացնում է ֆայլը commit-ի համար:
<code>git commit -m "msg"</code>	Պահպանում է փոփոխությունները:
<code>git log</code>	Ցույց է տալիս commit-ների պատմությունը:
<code>git branch</code>	Ցույց է տալիս ճյուղերը:
<code>git checkout <branch></code>	Փոխում է գործող ճյուղը:
<code>git merge <branch></code>	Միացնում է ճյուղերը:

ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Տեղադրի՛ր Git և ստուգի՛ր տարբերակը:
`git --version`
2. Սահմանի՛ր անունն ու email-ը:
`git config --global user.name "Քո Անունը"`
`git config --global user.email "example@gmail.com"`
3. Ստեղծի՛ր թղթապանակ և բացի՛ր Git պահոց դրա մեջ:
`mkdir my_project`
`cd my_project`
`git init`
4. Ստեղծի՛ր նոր ֆայլ, ավելացրո՛ւ Git-ի staging area-ում և commit արա՛:
`echo "Hello Git" > file.txt`
`git add file.txt`
`git commit -m "Ավելացրեցի առաջին ֆայլը"`
5. Ստուգի՛ր commit-ների պատմությունը:
`git log`
6. Ստեղծի՛ր նոր ճյուղ (branch) և գնա՛ դեպի նոր branch:
`git branch feature1`
`git checkout feature1`
7. Փոփոխի՛ր ֆայլը, commit արա՛ նոր ճյուղում, ապա միավորի՛ր main-ի հետ:
8. Կապի՛ր տեղական պանակը GitHub-ի հետ և push արա՛:
9. Անցի՛ր commit-ների պատմության մեջ և վերադարձի՛ր նախորդ տարբերակին:

Գրականություն և արտաքին հղումներ

Python 3 documentation: docs.python.org/3/

GeeksforGeeksonline platform <https://www.geeksforgeeks.org/>

W3 school python tutorial: w3schools.com/python/default.asp

Python in Education: education.python.org/resources/resource/list

Real Python Tutorials: realpython.com

Python documentation -<https://docs.python.org>

Research is Fun: research1.fun/category/python

PYnative Exercises: pynative.com/python-exercises-with-solutions

Matplotlib library <https://matplotlib.org/>

Pandas library <https://pandas.pydata.org/>

Numpy library <https://numpy.org/>

Seaborn library <https://seaborn.pydata.org/>

Plotly library <https://plotly.com/>

PEP 8 – Style Guide <https://peps.python.org/pep-0008/>

Wes McKinney “Python for Data Analysis”



Generation AI

Nurturing Future Innovators

